



Research Article

A systematic literature review on security testing of Ethereum smart contracts

Marwa Mnasri^{*}, Afef Jmal Maâlej^{ID,*}, Mohamed Jmaiel

ReDCAD Laboratory, National School of Engineers of Sfax, University of Sfax, BP 1173, Sfax 3038, Tunisia

ARTICLE INFO

Keywords:

Security testing
Smart contract vulnerabilities
Blockchain security
Ethereum smart contracts

ABSTRACT

The security testing of Ethereum smart contracts has become increasingly important with the rise of decentralized applications and blockchain technology. This systematic literature review (SLR) aims to provide a comprehensive overview of the state-of-the-art techniques, methodologies, tools, and challenges in the security testing of Ethereum smart contracts. By synthesizing and analyzing existing research articles, conference papers, and other relevant sources, this SLR identifies common trends, gaps, and areas for future research in this domain. The review covers various aspects of security testing, including vulnerability detection, testing frameworks, automated analysis tools, and best practices. In addition, it explores the impact of security vulnerabilities on smart contract ecosystems and proposes recommendations to improve the effectiveness and efficiency of security testing processes. This SLR serves as a valuable resource for researchers, practitioners, and developers interested in improving the security and reliability of Ethereum smart contracts.

1. Introduction

Smart contracts are revolutionizing the way agreements and transactions are executed in the digital world. Imagine a real estate transaction between Alice and Bob using smart contracts on the Ethereum blockchain. Traditionally, this process would involve multiple intermediaries, paperwork, and verification steps, leading to delays and increased costs. With smart contracts, Alice and Bob can automate the entire process [1].

1.1. Background and motivation

The emergence of smart contracts on blockchain platforms like Ethereum has ushered in a new era of trustless and efficient transactions. These contracts, powered by immutable code and distributed consensus mechanisms, offer unparalleled security and transparency compared to traditional agreements. However, these advantages come with challenges, particularly concerning ensuring the robustness and security of smart contract code. Addressing these challenges through rigorous testing, auditing, and continuous research is essential to fully exploit the potential of smart contracts and foster the widespread adoption of blockchain technology across various sectors [2].

1.1.1. Smart contracts and Ethereum

Smart contracts are computer programs stored on the Ethereum blockchain. They are executed automatically and transparently, without the need for an intermediary. This revolutionary technology offers several advantages:

- **Enhanced security:** smart contracts are immutable and transparent, making them more resistant to fraud and manipulation.
- **Increased efficiency:** smart contracts automate manual processes, saving time and reducing costs.
- **Improved accessibility:** smart contracts are accessible to everyone, regardless of their location or background.

In the context of Ethereum, smart contracts play a crucial role in creating decentralized applications (dApps). dApps are applications that run on the Ethereum blockchain and are not controlled by a single entity. They offer a wide range of services, such as decentralized finance (DeFi), gaming, and marketplaces [3].

1.1.2. Challenges and risks of smart contract security

Despite its advantages, smart contract technology also presents significant risks. Smart contracts are complex software that can be susceptible to vulnerabilities. Hackers can exploit these vulnerabilities to steal funds, manipulate data, or disrupt the functioning of smart contracts [4]. The stakes involved in smart contract security are high. Financial

^{*} Corresponding author.

E-mail addresses: marwa.mnasri@redcad.org (M. Mnasri), afef.jmal@redcad.org (A.J. Maâlej), mohamed.jmaiel@redcad.org (M. Jmaiel).

<https://doi.org/10.1016/j.bcr.2025.100314>

Received 6 May 2024; Received in revised form 2 February 2025; Accepted 2 February 2025

losses due to smart contract attacks amount to billions of dollars [5,6]. Moreover, attacks on smart contracts can negatively impact user confidence and hinder the adoption of blockchain technology [7].

1.1.3. Need for a systematic literature review (SLR)

Given the challenges and risks associated with smart contract security, it is crucial to comprehensively understand the techniques and tools available for testing and securing smart contracts. An SLR is a valuable tool for addressing this need. An SLR is a rigorous and scientific method for identifying, analyzing, and synthesizing existing research on a given topic. In the context of smart contract security, an SLR is necessary to provide a comprehensive and up-to-date understanding of this critical research area. This SLR will contribute to improving smart contract security and promoting the adoption of blockchain technology.

Software testing is an essential practice to ensure the quality, reliability, and security of computer applications. In the context of smart contracts, where trust and security are paramount, testing takes on particular importance. Security testing aims to identify potential vulnerabilities in the code of smart contracts and ensure that they function as expected, without risks of flaws or compromises.

Several aspects of security testing are relevant to smart contracts, including:

- **Static analysis:** this method examines the source code of smart contracts without executing them, identifying potential errors such as logic vulnerabilities or security flaws.
- **Dynamic analysis:** in contrast to static analysis, dynamic analysis involves executing smart contracts in a controlled environment to detect vulnerabilities in real time, such as stack overflow attacks or denial-of-service attacks.
- **Security audits:** security audits are in-depth examinations of smart contract code conducted by cybersecurity experts. These audits aim to identify and correct vulnerabilities before deploying smart contracts on the blockchain.

By integrating rigorous security testing practices into the development and deployment of smart contracts, developers can reduce the risks of exploitation by malicious actors and enhance user confidence in the smart contract ecosystem [8].

1.2. Objectives and research questions

The security of smart contracts on the Ethereum platform represents a major challenge in the blockchain domain. This SLR aims to contribute to a comprehensive understanding of this critical area. Furthermore, by identifying future research needs, this analysis aims to guide efforts to enhance the security of smart contracts on the Ethereum blockchain.

1.2.1. Main objectives

The primary objective of this SLR is to provide a comprehensive and in-depth understanding of security testing for Ethereum smart contracts. This SLR aims to:

1. Map existing approaches and techniques for security testing of Ethereum smart contracts.
2. Evaluate the effectiveness and limitations of each approach and technique.
3. Identify best practices for secure development and testing of smart contracts.
4. Determine future research needs in the field of security testing for smart contracts.

1.2.2. Research questions

To achieve the main objective, our SLR will address the following research questions:

- **RQ1:** What are the main security testing techniques for smart contracts used in the literature?

- **RQ2:** What are the primary tools and frameworks available for security testing of smart contracts?
- **RQ3:** What are the most common vulnerabilities identified in Ethereum smart contracts?
- **RQ4:** What metrics are used to evaluate the quality and effectiveness of security testing for Ethereum smart contracts?
- **RQ5:** What are the current challenges and limitations in the field of security testing for Ethereum smart contracts?

Research questions in an SLR play a crucial role in guiding the research process, identifying gaps in the literature, and addressing the needs of the scientific community.

1.3. Research methodology

SLRs are a cornerstone of research in many fields, including software engineering. They offer a comprehensive and objective way to evaluate existing research on a specific topic. This section explores a well-established approach for conducting SLRs: the Kitchenham method.

1.3.1. The Kitchenham method

The Kitchenham method [9] is a rigorous and well-established approach for conducting SLRs in the field of software engineering. It offers several advantages:

- **Rigor and reproducibility:** the Kitchenham method is based on scientific principles and provides a clear and documented process to ensure the rigor and reproducibility of the SLR.
- **Comprehensiveness and accuracy:** the methodology aims to identify and analyze all relevant primary studies for the SLR topic, minimizing the risk of bias and omissions.
- **Transparency and evaluation:** the Kitchenham method encourages detailed documentation of each step of the SLR, allowing for transparent assessment of the quality and validity of the research.

1.3.2. Stages of the Kitchenham methodology

The Kitchenham methodology consists of the following stages:

1. Planning the review

- Defining the objectives and research questions of the SLR.
- Developing a review protocol that describes the methodology and inclusion/exclusion criteria.
- Identifying potential sources of research (databases, libraries, etc.).

2. Conducting the review

- Identifying and selecting relevant primary studies.
- Extracting data from the selected studies.
- Analyzing and synthesizing the extracted data.

3. Reporting the review

- Presenting the results of the SLR in a clear and concise manner.
- Discussing the implications of the results and the limitations of the research.
- Formulating conclusions and recommendations for future research.

1.3.3. Details of key stages

- **Defining research questions and inclusion/exclusion criteria**
 - The research questions should be clear, precise, and relevant to the topic of the SLR.
 - Inclusion/exclusion criteria should be defined to identify relevant primary studies and exclude those that are not.
- **Identifying and selecting primary studies**
 - The search for primary studies should be conducted in multiple sources of research.
 - Primary studies should be selected based on the defined inclusion/exclusion criteria.
- **Data extraction and analysis**

- Relevant data should be extracted from the selected primary studies.
- The extracted data should be analyzed methodically to answer the research questions.
- **Synthesis and interpretation of results**
 - The results of the data analysis should be synthesized and interpreted in a coherent manner.
 - The conclusions of the SLR should be drawn from the synthesized results.

1.3.4. Conclusion

The Kitchenham methodology provides a robust framework for conducting a rigorous and high-quality SLR. By following its steps and documenting each stage in detail, researchers can ensure the production of a reliable and useful SLR that will contribute to the advancement of knowledge in the field of software engineering.

2. Literature review

In this section, we will discuss the inclusion and exclusion criteria used to select relevant studies for understanding Ethereum smart contract security testing. These criteria encompass a wide range of study types, methodologies, and topics to ensure a comprehensive analysis. The rigorous search process, following the Kitchenham methodology, was employed to identify potential sources and define relevant keywords. The application of inclusion and exclusion criteria resulted in the selection of 26 high-quality articles for the literature review, contributing to a better understanding of Ethereum smart contract security testing.

2.1. Inclusion and exclusion criteria

• Relevant study types

To gain a comprehensive understanding of Ethereum smart contract security testing, a wide range of studies is necessary, covering theoretical, practical, and empirical aspects:

- Empirical research evaluating security testing methods.
- Theoretical analyses exploring best testing practices.
- Case studies detailing specific smart contract vulnerabilities and the testing approaches used to identify them.
- Research on vulnerability detection methods using innovative approaches, such as machine learning.

• Inclusion criteria

To ensure relevance, quality, and accessibility of information, the following criteria will be applied:

- Language: English only.
- Publication period: From 2020 onwards.
- Topic: Specifically focused on Ethereum smart contract security testing.
- Methodology: Details specific security testing techniques, static or dynamic analysis frameworks, or innovative approaches for vulnerability detection (machine learning, etc.).

• Exclusion criteria

The following studies will be excluded:

- Irrelevant topics (e.g., blockchain security in general, outside Ethereum).
- Languages other than English.
- Published before the chosen start period.
- Not specifically focused on Ethereum smart contract security testing.
- Lacking a clear testing methodology.

2.2. Study selection

• Search process

The search for relevant studies followed the Kitchenham methodology in several steps:

1. Identification of potential sources

- Academic databases: ACM Digital Library, IEEE Xplore, ScienceDirect, SpringerLink.
- Scientific search engine: Google Scholar.

2. Definition of keywords

- “Ethereum smart contract security testing”
- “Ethereum smart contract vulnerabilities”
- “Smart contract security analysis”
- “Smart contract vulnerability detection”
- (And other similar terms)

3. Application of inclusion and exclusion criteria

- Out of the initially identified 160 articles on threat detection and testing tools (whether for vulnerable contracts only, malicious contracts only, or malicious and vulnerable contracts), 133 used methods such as static and dynamic analysis, fuzzing, mutation testing, and metamorphic testing. The remaining 27 articles explored machine learning for these detection and testing tasks.
- After filtering based on inclusion/exclusion criteria, 26 articles were selected for the literature review.

• Justification for filtering

Excluded articles did not meet the inclusion criteria, for example:

- Published before the chosen start period.
- Tools for detecting malicious contracts.
- Irrelevant topics (e.g., detection and testing tools without a specific focus on testing).
- Lack of clarity on the testing methodology applied to Ethereum smart contracts.

2.3. Selection results

This rigorous selection process, based on the Kitchenham method, ensures the inclusion of relevant and high-quality studies. It will lead to a robust analysis and synthesis of data, contributing to a better understanding of Ethereum smart contract security testing.

In this section, we will address the main aspects of Ethereum smart contract security testing as explored in the literature. The overall objective is to answer the main research questions. By examining these questions, we aim to provide a comprehensive overview of the current state-of-the-art in security testing methodologies, tools, vulnerabilities, and evaluation metrics specific to Ethereum smart contracts. This analysis will shed light on the advancements, challenges, and best practices in ensuring the security and reliability of smart contracts deployed on the Ethereum blockchain.

2.4. RQ1: What are the main security testing techniques for smart contracts used in the literature?

In this section, we embarked on extracting relevant data from each selected study, focusing on smart contract security testing. Our goal was to gather crucial information for our analysis, including the testing techniques used, the names of tools employed, study references, the number of vulnerabilities detected, the testing method, and links to the code repositories of free tools. This approach allows us to obtain a robust and diversified dataset, essential for a comprehensive and in-depth evaluation of security testing practices in the smart contract domain, as illustrated in Table 1.

2.4.1. Dynamic security testing

The evolution of testing techniques in the field of computer science was notably depicted in a pie chart, as shown in Fig. 1. This symbolic circle, divided into percentages, reveals the significant trends in the distribution of work carried out in various testing methods. The technique of fuzzing emerged as the predominant one, accounting for a

Table 1
Dynamic security testing tools for smart contracts.

Technique	Ref.	Tool name	Number of vulnerabilities	Repository	Year	Testing method	Last update date (version)
Fuzzing	[10]	FunFuzz	8	FunFuzz_tool	2024	Grey box	May 2023
	[11]	xFuzz	3	xfuzz_tool	2022	White Box	-
	[12]	MagicMirror	not provided	-	2023	Black Box	-
	[13]	IR-Fuzz	8	-	2022	White Box	-
	[14]	ItyFuzz	4	ityfuzz_tool	2022	White Box	-
	[15]	SMARTIAN	9	SMARTIAN tool	2021	Grey Box	May 2024 (v1.0)
	[16]	DEPOSafe	1	-	2021	Grey Box	-
	[17]	Echidna	not provided	Echidna_tool	2020	Grey Box	January 2025 (v2.2.6)
	[18]	Etherolic	9	-	2020	Grey Box	-
	[19]	GasFuzzer	7	-	2020	Black Box	-
	[20]	EF/CF	3	EF/CF tool	2023	White Box	July 2023 (v1.1)
	[21]	RLF	6	-	2022	Black Box	-
	[22]	Harvey	2	-	2020	Grey Box	-
	[23]	ConFuzzius	10	ConFuzzius-tool	2021	Black Box	July 2022 (v0.0.2)
	[24]	EVMFuzzer	not provided	EVMFuzzer-tool	2019	Grey Box	May 2019
Mutation Testing	[25]	EtherFuzz	1	-	2020	White Box	June 2020
	[26]	SuMo	-	-	2020	White Box	-
Symbolic Execution	[27]	SMARTEST	4	-	2021	Grey Box	-
	[28]	WANA	3	WANA_tool	2020	White Box	March 2021
	[29]	RA	1	RA_tool	2020	Grey Box	September 2021
	[30]	SAILFISH	2	-	2022	Grey Box	-
	[31]	teEther	4	teEther_tool	2018	White Box	July 2021
	[32]	DefectChecker	8	DefectChecker tool	2021	White Box	December 2024
Metamorphic Testing	[33]	-	3	-	2023	Grey Box	-

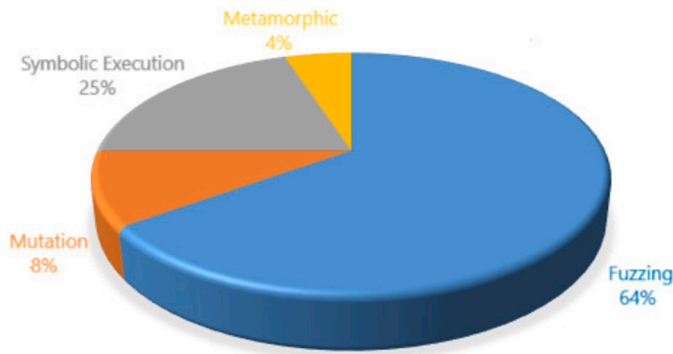


Fig. 1. Distribution of smart contract testing techniques.

significant portion of 65% of research and development efforts. This dynamic and exploratory approach has been extensively explored and exploited to identify flaws and vulnerabilities in software systems. Additionally, symbolic execution also occupied a substantial portion, totaling 20% of the work. This sophisticated method allowed for an in-depth analysis of possible program execution paths, thus providing a unique perspective on its robustness and reliability. Meanwhile, mutation testing represented a significant portion of 10%, underscoring its growing importance in validating code quality. Finally, although less common, metamorphic testing (MT) contributed 5% of the work, highlighting its promising potential in diversifying testing approaches to ensure the resilience of software systems against various and unforeseen scenarios. This pie chart thus reflects the diversity and dynamism of the landscape of testing techniques between 2020 and 2023, emphasizing the importance of ongoing research in this crucial field of computer science.

2.4.1.1. Fuzzing testing Fuzzing, also known as fuzz testing, is an automated software testing technique that involves providing invalid, unexpected, or random inputs to a computer program [11,13]. The goal is to detect bugs and vulnerabilities that traditional testing methods might not identify.

- **How it works**

A fuzzing tool generates random or semi-random data that matches the format expected by the input of the program to be

tested. It then injects these data into the program and monitors its behavior. If the program crashes, hangs, or exhibits unexpected behavior, it may indicate the presence of a bug or vulnerability.

- **Advantages**

- **Automation:** fuzzing allows for the automated testing of a large number of input cases, which is especially useful for complex programs with many possible inputs.
- **Effectiveness:** fuzzing can identify bugs and vulnerabilities that traditional testing methods, such as unit testing or integration testing, might not detect.
- **Code coverage:** fuzzing can explore code execution paths that are not always reached by traditional testing methods, which can help improve code coverage.

- **Disadvantages**

- **False positives:** fuzzing can generate input cases that do not cause real bugs, which can lead to false positive alerts.
- **Execution time:** fuzzing can be time-consuming, especially for complex programs with many possible inputs.
- **Required skills:** setting up and using fuzzing tools can require specific skills.

- **Types of fuzzing**

- **Black-box fuzzing:** this approach does not require any knowledge of the source code of the program to be tested. The fuzzing tool generates random or semi-random data based on the format expected by the program's input.
- **White-box fuzzing:** this approach uses information from the program's source code to generate more precise input cases.
- **Mutation-based fuzzing:** this approach generates input cases by modifying existing valid input cases.

- **Fuzzing tools**

- **Generic fuzzers:** these tools can be used to test different types of programs.
- **Specific fuzzers:** these tools are designed to test specific types of programs, such as web browsers or web applications.

2.4.1.2. Symbolic execution Symbolic execution [28] is a static software analysis technique that explores the different execution paths of a program by using symbolic values instead of concrete values. Unlike concrete execution, which only follows one possible path, symbolic ex-

ecution explores all possible paths, considering the different values that variables can take.

- **How it works**

Symbolic execution works by instrumenting the program's source code and simulating it step by step. At each step, the values of the variables are represented by symbolic expressions. The program instructions are then interpreted, and the symbolic expressions are updated according to the operations performed.

- **Advantages**

Symbolic execution has several advantages over other static analysis techniques:

- It can test all execution paths of the program, including those that are rarely taken.
- It can detect bugs that are not detectable by concrete execution, such as buffer overflows and divisions by zero.
- It can be used to prove properties of the program, such as termination or correctness.

- **Limitations**

Symbolic execution also has some limitations:

- It can be time-consuming and resource-intensive because it explores all execution paths of the program.
- It can be difficult to implement for complex programs.
- It may not be able to detect all bugs.

- **Additional information**

- Symbolic execution is a powerful tool for finding bugs in software, but it is not a silver bullet. It should be used in conjunction with other testing techniques to ensure that software is as bug-free as possible.
- Symbolic execution is a research area, and new techniques are being developed all the time to improve its effectiveness and efficiency.

2.4.1.3. Mutation testing Mutation testing [34] is a software testing technique that involves the intentional injection of modifications, called “mutations”, into the source code of a program. The goal is to determine whether existing tests are capable of detecting these mutations and ensuring the robustness of the code.

- **How it works**

1. **Mutant generation:** specific tools generate mutants by modifying the source code in different ways. Mutations can be simple, such as changing a constant value or a logical operator, or more complex, such as deleting a condition or a loop.
2. **Test execution:** existing tests are then executed on the original program and on each mutant.
3. **Result analysis:** mutation testing is considered successful if all mutants are detected by at least one test. If a mutant is not detected, it means that there is a missing test case and that the code is vulnerable to an error.

- **Advantages**

- **Improved test coverage:** mutation testing helps identify parts of the code that is not sufficiently tested and fills coverage gaps.
- **Detection of hard-to-find bugs:** mutation testing can identify bugs that are not detected by other testing techniques, such as unit tests or integration tests.
- **Improved code quality:** mutation testing encourages developers to write more robust and error-resistant code.

- **Limitations**

- **High cost:** mutation testing can be expensive in terms of time and resources, as it requires the generation and execution of a large number of mutants.
- **Difficult to interpret results:** it can be difficult to determine whether an undetected mutant represents a real bug or a harmless mutation.

- **Not applicable to all types of programs:** mutation testing is more effective for programs whose source code is accessible and easy to modify.

Example

Suppose we have the following Python code:

```
function(x):
    if x > 0:
        return x
    else:
        return -x
```

A mutation testing tool could generate the following mutants:

- Mutation 1: Change the > operator to >=.
- Mutation 2: Change the == operator to !=.
- Mutation 3: Delete the else condition.

If the existing tests do not detect all of these mutants, it reveals a gap in test coverage, indicating that the test suite may be incomplete or insufficiently robust. This exposes the code to potential errors similar to the undetected mutants.

2.4.1.4. Metamorphic testing MT [33] is a property-based software testing technique that can be an effective approach to solving the test oracle problem and the test case generation problem.

- **Principle**

MT is based on the idea that certain properties (called metamorphic relations (MRs)) must be satisfied by the system regardless of the input dataset.

Example

Consider a program that implements the sine function with a precision of 100 significant digits. An MR for the sine function is:

$$\sin(\pi - x) = \sin(x) \quad (1)$$

- **Process**

1. **Define MRs:** identify the properties that the system must satisfy after a transformation of the input data.
2. **Generate test cases:** apply MR transformations to existing test cases or generate new test cases.
3. **Execute test cases and compare results:** compare the obtained results with the expected results based on the MRs.
4. **Analyze results:** if the results do not match, it may indicate a defect in the system.

- **Advantages**

- Reduces the need for test oracles.
- Helps to generate more effective test cases.
- Detects that might not be detected by other testing techniques.

- **Limitations**

- Depends on the ability to identify meaningful MRs.
- Can be difficult to automate.

In conclusion, MT offers a valuable approach to software testing, particularly when dealing with challenges like test oracles and case generation. While it requires identifying meaningful MRs, the benefits of reduced oracle dependence, improved test case effectiveness, and defect detection outweigh these limitations. By leveraging this technique, developers can enhance the overall quality and reliability of their software systems.

2.4.2. Static security testing

2.4.2.1. Formal verification Formal verification [35] is a rigorous method used in software and hardware development to prove or disprove a system's compliance with formal specifications, which are mathematically defined properties. Unlike traditional testing methods, which check the proper functioning of a system through specific scenarios, formal verification examines all possible states of the system to ensure it

meets the given properties. Two of the most commonly used techniques in formal verification are model checking and theorem proving.

a. Model checking:

Model checking is an automated method of formal verification that involves exhaustively exploring the state space of a system model to check whether it satisfies certain specified logical properties, often expressed in terms of temporal logic, such as computation tree logic or linear temporal logic.

- **Process:** the model checking process involves three main elements:
 - **Modeling:** the system to be verified is represented as a formal model, such as a finite automaton, Petri net, or state transition system.
 - **Specification:** the properties to be verified are specified as logical formulas.
 - **Verification:** a model checking algorithm systematically explores the model's state space to determine if the specified properties are satisfied. If a violation is found, a counterexample is provided, indicating the sequence of states leading to the violation.
- **Advantages:**
 - **Exhaustiveness:** it verifies all possible states, ensuring complete coverage.
 - **Counterexamples:** in case of a violation, it provides a precise counterexample, facilitating the localization and correction of the issue.
- **Limitations:**
 - **State space explosion:** the number of states can grow exponentially with the system's size, making verification challenging for complex systems.

b. Theorem proving:

Theorem proving is another formal verification technique, based on using mathematical logic to prove that specified properties of a system are satisfied. Unlike model checking, which is automated and verifies a model through exhaustive exploration, theorem proving may require human intervention and is often applied to the verification of infinite systems or properties too complex to be handled by model checking.

- **Process:**
 - **Modeling:** the system is described by mathematical formulas in a formal logical language, often predicate calculus or constructive logic.
 - **Specification:** system properties are expressed as theorems to be proven.
 - **Proof:** the theorem proving tool, sometimes assisted by a human, attempts to prove the theorems as true using logical rules. This may involve automatic or semi-automatic deduction of new formulas until the theorem is proven or refuted.
- **Advantages:**
 - **Expressive power:** capable of handling systems with infinite states or complex properties that model checking cannot address.
 - **Precision:** allows for very detailed verification of systems, down to the level of mathematical logic.
- **Limitations:**
 - **Complexity:** may require significant expertise to correctly formulate theorems and to guide the proving tool.
 - **Less automation:** the process is often less automated than model checking, making it longer and more complex.

Formal verification is a powerful tool for ensuring that critical systems adhere to essential properties for their proper functioning. Model checking and theorem proving are two complementary tech-

niques that, despite each having advantages and limitations, contribute significantly to the reliability of complex systems across many domains.

2.4.2.2. Static code analysis

a. Abstract interpretation

Abstract interpretation is a static analysis technique that transforms source code into an abstract representation to evaluate its properties. It is based on creating an abstraction of the program, allowing exploration of possible states without executing the program concretely. Abstract interpretation for smart contracts involves creating a simplified version of the contract's source code to analyze its security properties.

- **Process:**
 - **Define abstractions:** identify and abstract key contract elements, such as variables and transaction states, to reduce complexity.
 - **Construct abstract model:** develop an abstract version of the contract code that captures critical behaviors while omitting irrelevant details.
 - **Analyze abstract model:** evaluate the model to detect potential issues, such as missing authorization checks or incorrect state transitions.
 - **Interpret results:** translate findings from the abstract model to guide potential improvements in the contract's security.
- **Advantages:**
 - **Efficiency:** allows for the analysis of code properties without requiring actual execution, saving time and resources.
 - **Scalability:** can handle large and complex codebases by focusing on abstract representations instead of concrete details.
 - **Early detection of security issues:** identifies possible vulnerabilities, such as unauthorized access or unintended state changes, before deployment.
- **Limitations:**
 - **Over-approximation:** abstract interpretation may yield false positives due to conservative assumptions, indicating issues that may not occur in practice.
 - **Complexity of abstraction:** constructing an effective abstract model for complex contracts can be challenging and may require expert knowledge.
 - **Limited precision:** the abstract model may miss certain detailed behaviors, potentially overlooking some issues that occur in specific states.

This method is particularly useful for uncovering general security properties but may require complementary techniques to achieve comprehensive assurance.

b. Pattern recognition

Pattern recognition (or pattern detection) involves searching for specific patterns or structures in the source code that may indicate bugs, vulnerabilities, or poor programming practices.

- **Process:**
 - **Define patterns to search for:** before analyzing the code, it is essential to identify potentially dangerous patterns. Some types of code could enable attackers to steal funds or manipulate a contract's outcomes.
 - **Analyze the smart contract code:** a specialized tool or program scans the contract code to search for these specific patterns, identifying code segments that resemble known problematic structures.
 - **Verify if the pattern is truly an issue:** once a pattern is detected, it's important to assess whether it's genuinely dangerous in this specific context, as cer-

tain patterns may not always be risky depending on their usage.

- **Reporting and fixes:** if the identified pattern is deemed dangerous, a report is generated explaining the issue and suggesting code modifications to correct it.

- **Advantages of pattern:**

- **Pre-deployment prevention:** detects security vulnerabilities before the contract is deployed, ensuring greater reliability.
- **Quick identification of common issues:** enables the detection of common vulnerabilities like reentrancy without the need for an exhaustive analysis.
- **Ease of automation:** can be automated to monitor code and instantly flag dangerous patterns.

- **Limitations of pattern recognition**

- **Reliance on known patterns:** only detects vulnerabilities based on previously identified patterns, limiting the detection of new types of vulnerabilities.
- **False positives:** may flag issues even if the detected pattern isn't genuinely dangerous in the specific context.
- **Need for complementary techniques:** for complete security coverage, other methods may be required to detect vulnerabilities that do not follow well-defined patterns.

Pattern recognition is a useful approach for identifying vulnerabilities in smart contracts by detecting potentially dangerous code structures before deployment. However, it is important to note that this method is not a true testing technique in the strict sense. It helps analyze known patterns but does not guarantee an exhaustive verification of the contract's behavior in all possible scenarios. Therefore, while effective in detecting certain vulnerabilities, pattern recognition should be used in conjunction with other formal and dynamic testing techniques to provide a comprehensive assessment of smart contract security.

c. Taint analysis

Taint analysis [36] is a dynamic security analysis technique used to track the flow of sensitive data within a program, such as inputs or state changes in smart contracts. The goal is to identify whether untrusted data affect critical variables, leading to vulnerabilities such as unauthorized access, data leakage, or improper state transitions in the contract. Taint analysis helps identify unintended interactions between untrusted inputs and sensitive operations in smart contracts, especially in scenarios involving user input, external calls, or interactions with other contracts.

- **Process**

- **Mark sensitive data:** the process starts by labeling the input data or variables of interest as “tainted.” These could be user inputs or external data that could potentially influence the contract's state.
- **Track data propagation:** as the contract executes, the tainted data are tracked through all operations, ensuring that any interaction with critical variables, such as account balances or function calls, is examined.
- **Identify dangerous interactions:** when tainted data influence sensitive operations, the analysis flags these interactions as potentially dangerous, indicating that malicious data could exploit the system.
- **Report findings:** finally, the analysis produces a report detailing the flow of tainted data and highlighting any potentially dangerous paths or vulnerabilities, recommending fixes.

- **Advantages:**

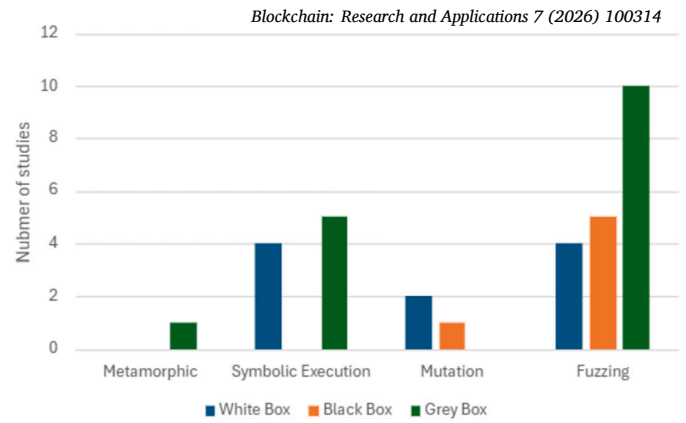


Fig. 2. Distribution of testing techniques by test box type for smart contract security.

- **Effective detection of data propagation issues:** taint analysis is particularly useful for tracking the flow of untrusted data through a smart contract and identifying vulnerabilities related to improper data handling.
- **Dynamic analysis:** unlike static analysis, taint analysis operates on actual contract execution, which allows it to detect runtime issues that might not be apparent in source code alone.
- **Detection of complex vulnerabilities:** it helps identify subtle vulnerabilities that can arise due to interactions between user input, external calls, or internal contract states.

- **Disadvantages:**

- **Overhead and performance cost:** taint analysis can introduce performance overhead, especially in large or complex contracts, as it tracks the flow of data during execution.
- **False positives:** it may flag benign interactions as vulnerable if not configured correctly, requiring manual review to filter out irrelevant findings.
- **Limited scope:** while useful for detecting data-related vulnerabilities, taint analysis may not fully address other types of security issues, such as logic flaws or incorrect state transitions.

Taint analysis is a valuable technique for identifying security vulnerabilities in smart contracts, particularly when examining data flows and potential sources of malicious input. However, it is not a testing tool on its own, but rather a method that should be integrated into a broader testing strategy. To achieve comprehensive security assurance, it must be complemented by other techniques, as it primarily focuses on tracking data dependencies and may not detect all types of vulnerabilities, such as logical or architectural flaws in the contract's design. Thus, while effective in many scenarios, taint analysis should be used in conjunction with additional testing methods to ensure the overall security of smart contracts.

3. RQ2: What are the primary tools and frameworks available for security testing of smart contracts?

For security testing of smart contracts, a variety of tools and frameworks are available, each offering distinct approaches to identify and mitigate potential vulnerabilities. Fig. 2 presents a breakdown of tools across three testing categories: white, black, and grey box, along with various testing techniques such as fuzzing, mutation testing, symbolic execution, and MT.

Fuzzing, a technique that involves generating random input data to detect errors randomly, is widely used in security testing of smart con-

tracts. We observe that nine tools are specifically designed for fuzzing in the context of grey-box testing, indicating a growing interest in this approach combining elements of both white-box and black-box testing.

Mutation testing, which involves modifying the contract's source code to assess the robustness of tests, is less common. Only two tools are listed for white-box testing in Fig. 2. This may suggest that mutation testing is considered less of a priority or less applicable in the specific context of smart contracts.

Symbolic execution, a technique that explores all possible execution paths of a program, is also represented, but to a lesser extent than fuzzing. Two specific tools are listed for white-box testing and five for grey-box testing. This may indicate that symbolic execution is used more selectively, perhaps due to its complexity or resource requirements.

Lastly, MT, which verifies whether a transformation of valid input preserves the expected property, is less commonly used, with only one tool listed for grey-box testing. However, this does not necessarily mean that this technique is less important, but rather that it may be less developed or less known in the specific context of smart contracts.

In conclusion, Fig. 2 highlights the diversity of approaches to security testing of smart contracts, with a strong interest in fuzzing and variability in the adoption of other techniques such as mutation testing, symbolic execution, and MT. This analysis underscores the importance of carefully selecting tools and techniques based on the specific needs of the project and the characteristics of the smart contract being tested.

3.1. Fuzzing testing

A) FunFuzz

FunFuzz is a dedicated function-focused fuzzer that enhances vulnerability detection in smart contracts through its targeted strategies and efficient exploration methods, making it a valuable asset in the field of smart contract security.

- **Main features of FunFuzz**

Function-focused fuzzing: FunFuzz focuses on identifying and testing risky functions within smart contracts rather than attempting to cover all code indiscriminately. This targeted approach increases the likelihood of discovering vulnerabilities.

Unique vulnerability detection strategies: the tool employs three specific strategies aimed at detecting common vulnerabilities in smart contracts, such as reentrancy, block dependency, and gasless send. This specialization allows for more efficient vulnerability identification.

Exploration efficiency: by excluding non-risky functions, FunFuzz significantly reduces the scope of exploration during fuzzing, enabling more efficient testing compared to traditional fuzzers.

- **Advantages of FunFuzz**

High efficiency: FunFuzz has been shown to outperform state-of-the-art fuzzers in terms of efficiency and effectiveness when tested on 170 real contracts, making it a reliable tool for smart contract security testing.

Tailored for smart contracts: the tool is specifically designed for the unique characteristics and vulnerabilities of smart contracts, making it more suitable than general-purpose fuzzers in this field.

Time-saving: by focusing on risky functions and reducing the exploration of irrelevant code, FunFuzz saves time during the testing process, enabling developers to identify vulnerabilities more quickly.

- **Limitations of FunFuzz**

Vulnerability scope: while FunFuzz is effective for certain types of vulnerabilities, it may not cover all possible smart contract vulnerabilities. The focus on specific functions means that some issues could potentially be overlooked.

Reliance on function identification: FunFuzz's effectiveness largely depends on accurately identifying risky functions. If the identification process is flawed, some vulnerabilities might go undetected.

- **Future work for FunFuzz**

Expanding vulnerability coverage: future developments could focus on broadening the range of vulnerabilities FunFuzz can detect, potentially incorporating more advanced techniques to identify additional risks in smart contracts.

Integration with other tools: FunFuzz could be improved by integrating with other security analysis tools to provide a more comprehensive security assessment for smart contracts.

User feedback and adaptation: collecting user feedback and adapting the tool based on real-world use could help enhance its functionality and effectiveness in detecting vulnerabilities across various smart contract scenarios.

In summary, FunFuzz offers a promising approach to smart contract vulnerability detection with its targeted strategies and efficiency, while facing some limitations that could be addressed in future iterations.

B) MagicMirror

MagicMirror [12] is a powerful tool designed for testing smart contracts. Leveraging constraint resolution, selective state exploration, and combinatorial testing, it aims to enhance code coverage and mitigate state explosion issues. Below are the main features, advantages, and limitations of MagicMirror, along with potential future enhancements.

- **Main features**

- Fuzzing tool for smart contracts: MagicMirror specializes in testing smart contracts, which are self-executing contracts with the terms of the agreement directly written into code. It employs various techniques to generate test data and explore contract behaviors.
- Utilization of constraint resolution: MagicMirror employs constraint resolution algorithms to generate test inputs that satisfy predefined conditions within smart contracts.
- Selective state exploration: the tool selectively explores different states of the smart contract to identify relevant execution paths and maximize code coverage.
- Combinatorial testing: MagicMirror conducts combinatorial tests to systematically evaluate interactions between different parameters of smart contracts, thereby detecting potential errors related to these interactions.
- Large-scale validation: MagicMirror has been evaluated using over 2000 smart contracts, demonstrating its effectiveness in achieving high code coverage and detecting vulnerabilities across a wide range of use cases.

- **Advantages**

- High code coverage: MagicMirror effectively achieves high code coverage in smart contracts, which is crucial for ensuring their reliability and security.
- Vulnerability detection: by employing advanced fuzzing and combinatorial testing techniques, MagicMirror can efficiently detect vulnerabilities in smart contracts, enabling developers to address them before they are exploited.

- **Limitations**

- The effectiveness of MagicMirror in achieving high code coverage and detecting vulnerabilities may depend on the complexity and size of the tested smart contracts.

- The tool's performance and efficiency can vary depending on the specific use case and the number of contracts evaluated.
- Although MagicMirror utilizes constraint resolution and selective state exploration to mitigate the problem of state explosion, it may still face limitations in handling extremely large state spaces.
- While the document does not explicitly mention any other specific limitations or constraints of the MagicMirror tool, as with any software tool, there may be potential limits related to compatibility, scalability, and the ability to handle new or unknown vulnerabilities.

• **Future works** While the article does not explicitly outline MagicMirror's future work, several areas for improvement can be considered:

- **Enhanced vulnerability detection:** MagicMirror could be improved to detect specific types of vulnerabilities commonly found in smart contracts, such as reentrancy attacks or permission management issues.
- **Handling more complex smart contracts:** as smart contracts become more intricate, optimizing MagicMirror to effectively handle these advanced use cases would be beneficial.
- **Performance and efficiency optimization:** there is always room to optimize MagicMirror's performance to reduce the time required for test execution and result analysis, making the tool faster and more convenient for developers to use.

In summary, future work could focus on improving vulnerability detection, adapting to more complex smart contracts, and optimizing the tool's performance.

C) IR-Fuzz

IR-Fuzz [13] stands out as a fuzzing tool specifically designed for smart contracts, integrating innovative features and techniques to enhance the detection of errors and vulnerabilities.

• Main features

- **Invocation scheduling fuzzing:** IR-Fuzz excels in generating sequences of function invocations by considering data dependencies between functions, enabling the exploration of richer states. It extends the fuzzing process by adding new sequences from different states, going beyond the initial state.
- **Branch distance-based measurement:** IR-Fuzz incorporates a branch distance-based measurement to iteratively evolve test cases towards a target branch. This guides the fuzzing process to unexplored branches, increasing the chances of detecting bugs triggered by different states.
- **Discovery of rare and vulnerable branches:** IR-Fuzz uses a branch search algorithm to identify rare and vulnerable branches. By focusing on these crucial branches, IR-Fuzz optimizes resource allocation and avoids wasting energy on normal branches.
- **Higher branch coverage and vulnerability detection:** empirical results show that IR-Fuzz achieves 28% higher branch coverage compared to state-of-the-art fuzzing approaches. It also detects more vulnerabilities and increases the average precision of vulnerability detection by 7% compared to current methods.
- **Seed optimization paradigm:** IR-Fuzz employs a seed optimization paradigm that utilizes branch distance-based measurement to evolve test cases towards a target branch. This helps reduce the ran-

domness in test case generation and enhances fuzzing efficiency.

• Advantages

- **Effective vulnerability detection:** IR-Fuzz excels in detecting various vulnerabilities, significantly surpassing other state-of-the-art tools in comparative experiments. Its ability to identify critical flaws provides robust protection against potential threats.
- **Better branch coverage:** IR-Fuzz's approach distinguishes itself by achieving high branch coverage, especially on large contracts. This feature positions it ahead of competitors, ensuring comprehensive exploration of the code.
- **Efficiency and speed:** IR-Fuzz stands out for its ability to generate a large number of test cases in a relatively short period. Its execution speed positions it as an efficient tool, saving time for security researchers.
- **Discovery of new vulnerabilities:** IR-Fuzz is capable of detecting unique vulnerabilities not addressed by other existing tools. This reveals its ability to anticipate and address novel flaws in smart contracts.
- **Open source code:** the availability of IR-Fuzz's source code on GitHub promotes transparency, collaboration, and community contribution. This openness enhances user trust and encourages continuous improvement of the tool.

• Limitations

However, some drawbacks should also be considered:

- **Calculation of priority order:** the generation of invocation sequences based on priority order calculation may introduce computational overhead, especially with numerous read and write operations on the global variables.
- **Seed mutation optimization:** the current approach may generate repetitive and invalid test cases, suggesting opportunities for optimization to avoid mutating critical parts of test inputs.
- **Limitations in complex vulnerability detection:** while effective, IR-Fuzz may have limitations in detecting complex vulnerabilities that require in-depth analysis.
- **Configuration complexity:** setting up IR-Fuzz can be complex, especially when adjusting parameters related to energy allocation and other mechanisms.

In summary, IR-Fuzz emerges as a powerful tool for smart contract fuzzing, but careful configuration and a deep understanding of its mechanisms are necessary to fully leverage its capabilities.

• Future works

For future developments, substantial improvement avenues have been identified:

- A comprehensive analysis of sequence generation, aimed at optimizing the calculation of function call priority, is recommended.
- Additionally, seed mutation optimization is recognized as a major focus, emphasizing advanced strategies to reduce the generation of repetitive test cases.
- Special attention is given to user interface enhancements for improved accessibility, and efforts to integrate with popular development environments are recommended to facilitate adoption by developers.
- Exploration of dynamic analysis techniques, effective management of large-scale contracts, and broadening vulnerability coverage are also highlighted as key areas for improvement.

- Inter-blockchain compatibility is considered, allowing IR-Fuzz to adapt to other blockchain platforms.
- Lastly, continuous performance optimization, including resource management and parallel processing, is advocated to ensure the overall efficiency of IR-Fuzz.

In conclusion, the future evolution of IR-Fuzz relies on a holistic approach, combining technical enhancements, improved user interaction, expanded compatibility, and adaptability to evolving blockchain landscapes. Community involvement and an iterative development approach will be crucial to maintain the continuous relevance and effectiveness of this essential tool in the dynamic field of smart contract security.

D) ItyFuzz

Meeting the imperative of robust testing mechanisms, ItyFuzz [14] stands out as an innovative solution by introducing a snapshot-based approach, thereby redefining classical fuzzing methodologies.

• Main features

Let's delve deeply into the key features of ItyFuzz:

- **Snapshot-based approach:** ItyFuzz revolutionizes fuzzing by favoring the storage of snapshots over transaction sequences. This approach streamlines the exploration of input and program spaces, leading to a more resource-efficient testing process.
- **Data flow waypoint mechanism:** integrating a data flow waypoint mechanism enhances ItyFuzz's ability to discern states with significant momentum, thereby amplifying the overall efficiency of the fuzzer.
- **Comparison waypoints:** ItyFuzz utilizes comparison waypoints to judiciously prune the state space, optimizing the exploration process and contributing to the fuzzer's effectiveness in identifying vulnerabilities and potential exploits.
- **Exploit synthesis:** by preserving state snapshots, ItyFuzz demonstrates a remarkable ability to rapidly synthesize concrete exploits, even in complex scenarios such as reentrancy attacks. This quality underscores its effectiveness in identifying and characterizing vulnerabilities within smart contracts.
- **Second-level response time:** ItyFuzz boasts an impressive second-level response time, making it exceptionally suited for on-chain testing of smart contracts. This capability offers a practical advantage over local development testing, ensuring real-world applicability.
- **Surpassing existing fuzzers:** ItyFuzz excels in terms of instruction coverage and rapid generation of realistic exploits for on-chain projects. This notable achievement highlights its ease of use and effectiveness in targeting vulnerabilities within smart contracts.

• Advantages

Within the landscape of benefits that ItyFuzz brings to smart contract security, several highlights emerge:

- **Exceptional performance:** ItyFuzz has demonstrated exceptional performance in terms of instruction coverage, surpassing other tools such as SMARTIAN on various datasets.
- **Identification of real-world exploits:** the tool has succeeded in identifying concrete exploits for a large number of real-world projects, demonstrating its effectiveness in real-world vulnerability detection.
- **Fuzzing time optimization:** through its snapshot-based approach, ItyFuzz has significantly reduced fuzzing time on potentially benign contracts, saving up to 80% of time compared to other tools.

- **Prioritization of relevant paths:** the path prioritization strategy has enabled ItyFuzz to effectively focus on the most relevant paths, thereby improving vulnerability detection efficiency.
- **On-chain audit:** ItyFuzz has demonstrated its utility in on-chain auditing, uncovering overlooked vulnerabilities during the development cycle and providing a sufficiently fast response time to prevent exploits.
- **Adaptability to various data sets:** ItyFuzz has been successfully evaluated on different datasets, highlighting its adaptability to diverse environments, from ERC20 contracts to Ethereum projects.

• Limitations

In the thorough examination of drawbacks associated with ItyFuzz's usage, several critical considerations emerge, requiring a balanced evaluation of its limitations:

- **Limitations in handling complex conditions:** ItyFuzz has shown difficulties in managing conditions of complex paths involving multiple variables and values, sometimes leading to false negatives.
- **Need for integration with a theorem prover:** to overcome false negatives related to path complexity conditions, integration with a theorem prover like Z3 is suggested, adding additional complexity.
- **Memory overhead with ItyFuzz-Rand:** ItyFuzz-Rand ablation has revealed memory overhead (OOM) issues due to the continuous addition of states in the infant state corpus, requiring more efficient memory management.
- **Long vulnerability discovery time for some cases:** in complex situations, ItyFuzz may require more time to discover certain vulnerabilities, which may not be ideal for all deployments.
- **Dependency on waypoints:** ItyFuzz-DF, based on data flow waypoints, may sometimes miss crucial states, necessitating precise management of state addition to the corpus.

• Future works

- As part of the ongoing development of ItyFuzz, significant enhancements to the dataflow waypoint mechanism are being considered to refine the identification of states with substantial potential. This initiative aims to optimize the exploration process, ensuring increased efficiency in detecting vulnerabilities within smart contracts.
- Further exploration will involve evaluating the integration of additional pruning techniques. This approach aims to further fine-tune the space of states, thereby enhancing the efficiency of the fuzzing process and strengthening ItyFuzz's ability to precisely target various scenarios.
- A promising avenue to explore is the potential incorporation of machine learning techniques. This innovation could significantly enhance ItyFuzz's ability to identify interesting states and generate realistic exploits, providing an advanced perspective in proactive vulnerability detection.
- Comprehensive evaluations of ItyFuzz's effectiveness on a broader range of real-world smart contracts and on-chain DeFi projects will be conducted to gain in-depth insights into its performance in diverse situations. These evaluations will contribute to the continuous adjustment and improvement of ItyFuzz, ensuring its adaptation to the evolving landscape of smart contract vulnerabilities and security best practices.

- Active collaboration with the smart contract development community will remain a priority, allowing ItyFuzz to benefit from feedback and innovative ideas for ongoing improvements and refined adjustments, thus ensuring its excellence in smart contract security.

E) Deposafe

DEPOSafe [16] is an innovative tool designed to evaluate smart contracts for vulnerabilities related to fake deposit attacks. It follows a comprehensive approach involving both static analysis and dynamic validation to identify potential risks accurately. Below are the main characteristics, advantages, disadvantages, and future work aspects of DEPOSafe based on the provided passage:

• Main features

- Comprehensive approach: DEPOSafe combines the static detector and dynamic validator to thoroughly analyze smart contracts for fake deposit vulnerabilities.
- Static detector: utilizes symbolic execution based on Mythril to screen out vulnerable addresses in smart contracts by identifying potential fake deposit loop-holes.
- Dynamic validator: verifies flagged contracts through interactions with a local private Ethereum chain using web3, ensuring accurate detection of vulnerabilities.
- Detection logic: employs a systematic detection logic that includes enumerating standard APIs, locating key storage, and verifying required throws to identify vulnerable smart contracts.

• Advantages

- High precision: achieves high precision with a 100% reported vulnerable contracts rate, indicating accurate detection of fake deposit vulnerabilities.
- Comprehensive coverage: analyzes a significant number of ERC-20 smart contracts (176k) to provide a comprehensive overview of the fake deposit vulnerability landscape.
- Effective dynamic validation: the dynamic validator eliminates false positives introduced by static analysis, enhancing the accuracy of vulnerability detection.
- Impact measurement: measures the impact of vulnerabilities by analyzing the number of vulnerable tokens, market capitalization, and potential financial risks.

• Limitations

- False positives: static analysis may introduce false positives due to limitations in semantic information and inter-contract symbolic execution, leading to some inaccuracies in initial detection.
- False negatives: dynamic validation may result in false negatives when encountering smart contracts with non-standard requirements or insufficient initialization, impacting the completeness of detection.

• Future works

- Enhanced static analysis: improve static analysis techniques to reduce false positives by incorporating more semantic information and addressing inter-contract execution challenges.
- Dynamic validation enhancements: enhance dynamic validation to handle non-standard contract implementations and ensure thorough coverage of vulnerability detection.
- Cross-blockchain analysis: extend analysis beyond Ethereum to other blockchains (e.g., USDT, EO-

SIO) where fake deposit attacks have been observed, broadening the scope of vulnerability assessment.

- Real-world attack monitoring: develop capabilities to monitor real-world attacks and validate detection results against actual exploit attempts, enhancing the validation process's real-world relevance. Overall, DEPOSafe demonstrates effective detection capabilities for fake deposit vulnerabilities but requires ongoing improvements to address false positives/negatives and expand its coverage to diverse blockchain ecosystems. Future work should focus on refining analysis techniques, enhancing validation processes, and extending analysis to multiple blockchain platforms to ensure comprehensive and accurate vulnerability assessment.

F) SMARTIAN

SMARTIAN [15] offers an innovative approach to testing smart contracts.

• Main features

- SMARTIAN combines static and dynamic analyses to test smart contracts effectively.
- It operates in three main steps: INFOGATHER, SEEDPOOLINIT, and DATAFLOWFUZZ.
- INFOGATHER collects data on data flow within smart contracts.
- SEEDPOOLINIT predicts initial transaction sequences likely to lead to meaningful exploration.
- DATAFLOWFUZZ mutates initial seeds and evaluates their relevance for bug discovery.
- SMARTIAN uses data flow-based feedback to guide the fuzzing process and enhance test efficiency.
- It implements bug oracles to detect various types of bugs, contributing to test reliability and precise defect identification.

By integrating these functionalities, SMARTIAN offers a comprehensive and efficient approach to smart contract testing, thereby improving the quality and robustness of blockchain-based applications.

• Advantages

- High efficiency in bug detection, surpassing other tools in the benchmarks used.
- Broad and diverse coverage of smart contract use cases.
- Reduction of false positives through precise bug oracles.
- Low overhead in terms of execution time and resources.
- Open-source and reproducibility due to benchmark and code availability.

• Limitations

- Input data dependency: like any fuzzing tool, the quality of results largely depends on the input data provided. Inadequate test coverage for critical scenarios may lead to some bugs escaping detection.
- Configuration complexity: configuring SMARTIAN for specific use cases may require deep technical expertise, especially in adjusting static and dynamic analysis parameters.
- Storage overhead: generating numerous test cases and tracking data can lead to storage overhead, particularly for complex smart contracts and large datasets.
- Limitations in handling non-linear branches: SMARTIAN may encounter difficulties with non-linear branches, limiting its effectiveness in certain specific use cases.

- Continuous maintenance requirement: like any software, SMARTIAN requires ongoing maintenance to remain compatible with evolving smart contracts, blockchain platforms, and associated tools.

- **Future works and potential improvements**

- Enhance static analysis precision to reduce false positives.
- Integrate techniques to handle non-linear branch conditions.
- Explore extending SMARTIAN to other domains requiring critical scenario testing.
- Validate at scale across a variety of use cases to confirm its effectiveness and robustness.

In conclusion, SMARTIAN represents a significant advancement in the field of smart contract testing thanks to its combined approach of static and dynamic analyses. Its key features, such as data-flow-based feedback and precise bug oracles, make it a promising tool for enhancing the quality and reliability of blockchain applications. Despite some challenges to overcome, SMARTIAN has the potential to become an essential tool for developers and testers in the blockchain ecosystem, thereby contributing to building user trust and fostering wider adoption of blockchain technology.

G) Ethrolic

Ethrolic [18] stands out as a robust, scalable, and effective fuzzing tool, leveraging concolic testing for in-depth security analysis of Ethereum smart contracts. Its distinctive features elevate it to the status of a powerful and innovative tool in this domain.

- **Main features**

- **Combination of techniques:** Ethrolic combines dynamic data tracing and concolic testing to identify and exploit vulnerabilities in Ethereum smart contracts.
- **Written in Rust:** the tool is implemented in Rust (version 1.35.0), providing high performance and increased security.
- **Abstract architecture:** Ethrolic features an abstract architecture with components such as StateDB, the labeling engine, the attack indicator, the protection detector, the test generator, and the report generator.
- **Taint engine:** the Taint Engine assigns labels to data from untrusted sources, following the propagation of these labeled data in the target code during execution.
- **Attack indicator:** after identifying sensitive data flows, the attack indicator inspects labeled data to detect potential signs of attacks. It can request the cancellation of transactions responsible for these suspicious states.
- **Protection detector:** Ethrolic monitors the presence of protection methods in the analyzed code, reducing false positives. The protection detector assesses the impact of potential protection functions on the propagation of labeled data.
- **Test generation:** the test generator uses concolic tests and relies on the Manticore framework. It produces triggering transactions for vulnerable parts of the tested contracts.
- **Report generator:** Ethrolic generates detailed reports providing essential information about detected vulnerabilities, offering a clear view of the issues.

- **Advantages**

Ethrolic presents significant advantages that position it as a preferred tool for Ethereum smart contract security analysis. Firstly, its lack of source code requirements offers valuable flexibility, enabling its application to business contracts where the source code may not be accessible. This feature

distinguishes it by making its usage more widespread and suitable for real-world scenarios where source code confidentiality may be a concern. Another major advantage of Ethrolic lies in its recognition of protection methods embedded in the code. Unlike other tools, Ethrolic takes these methods into account, resulting in a significant reduction of false positives. This particular attention to protection measures enhances the quality of generated reports, providing a more precise and relevant analysis of potential vulnerabilities. The demonstration of Ethrolic's effectiveness is reinforced by tangible results. By identifying 204 real security violations among 98 protection features in the benchmark suite, Ethrolic proves its ability to detect real vulnerabilities, attesting to its relevance in real-world environments.

- **Limitations**

However, like any tool, Ethrolic also presents challenges and drawbacks. The complexity of protection methods can pose difficulties in their correct interpretation by security analyzers, potentially leading to false positives. The need to understand and address sophisticated protection strategies remains a challenge.

- **Challenges and future works**

A major challenge faced by Ethrolic lies in specifying new protection models, which can influence the accuracy of its results. To overcome this difficulty, exploring techniques based on machine learning represents a promising avenue. The integration of these methods could enhance Ethrolic's ability to adapt to emerging protection strategies and reduce false positives. Additionally, the discovery of new sophisticated attacks poses another potential challenge. Existing tools may be limited in their ability to identify such unknown attacks. The use of deep learning offers an opportunity for Ethrolic to explore and extract new attack patterns, thereby strengthening its ability to stay updated against emerging and complex threats.

In conclusion, while Ethrolic offers significant advantages, recognizing its current disadvantages and challenges is part of a perspective of continuous improvement, with a particular focus on exploring machine learning-based solutions to address future challenges.

H) EF/CF

Extremely fast contract fuzzer (EF/CF) [20] is a high-throughput fuzzing tool designed to identify vulnerabilities in smart contracts.

- **Main features**

- **Compile time optimization:** EF/CF translates Ethereum virtual machine (EVM) bytecode of the smart contract to C++ code using a custom compiler called `evm2cpp`. This compiled code is then paired with a fuzzing-optimized EVM runtime for faster execution compared to interpreter-based approaches.
- **Modeling complex interactions:** it allows fuzzing the behavior of multiple attacker-controlled smart contracts alongside the target contract. The fuzzer can define how these attacker contracts respond to calls from the target contract, enabling exploration of reentrancy attacks and other complex interaction issues.
- **Customizable bug oracles:** EF/CF primarily uses a simple yet effective Ether-gain bug oracle. This means it identifies attacks that allow the attacker to steal Ether from the target contract. Additionally, it supports custom bug oracles written in Solidity code by the developer to target specific vulnerabilities.
- **Structure-aware mutation:** EF/CF employs a custom mutator called `ethmutator` that can perform both mutations on the transaction inputs themselves (based on the smart contract's application binary in-

interface (ABI)) and structural mutations on the transaction sequence. This allows for more efficient exploration of the attack surface.

- **High throughput:** by combining compile-time translation, a fuzzing-optimized runtime, and efficient mutation strategies, EF/CF achieves high throughput in its fuzzing campaigns.

Overall, EF/CF is designed to thoroughly test smart contracts for vulnerabilities by simulating various attack scenarios and identifying potential weaknesses in their logic or interaction patterns.

- **Advantages**

- **Scalability:** it can handle longer transaction sequences, which is crucial for analyzing complex contracts.
- **Bug detection:** EF/CF can detect a wide range of vulnerabilities, making it a versatile tool for smart contract security.
- **Performance:** it shows performance comparable to symbolic execution while also generating exploitable real-world transactions.

- **Limitations**

- **Reentrancy:** the prototype version of EF/CF does not yet support contract creation at runtime, limiting its ability to detect certain types of reentrancy-related vulnerabilities.
- **Contract complexity:** although EF/CF is effective in detecting a wide range of bugs, it may encounter challenges with highly complex and unstructured contracts.

- **Future works**

- **Improving reentrancy detection:** developing methods to detect reentrancy in contracts, including support for runtime contract creation, could be an interesting avenue to enhance EF/CF's capabilities.
- **Scalability optimization:** continuing to optimize EF/CF to handle even longer transaction sequences while maintaining high performance would be beneficial for analyzing increasingly complex contracts.
- **Expanding code coverage:** exploring ways to increase EF/CF's code coverage to include more advanced aspects of static and dynamic smart contract analysis. In summary, EF/CF has promising capabilities in terms of bug detection in smart contracts, but there are still areas for improvement to handle more complex cases and to extend its code coverage.

I) ConFuzzius

ConFuzzius [23] is an advanced hybrid tool designed for evaluating and detecting vulnerabilities in smart contracts deployed on blockchains. It combines state-of-the-art symbolic execution and fuzzing techniques to enhance code coverage and improve vulnerability detection. In this section, we will delve into the main characteristics, advantages, disadvantages, and future work of ConFuzzius.

- **Main features**

- **Advanced symbolization and fuzzing methods:** ConFuzzius integrates cutting-edge techniques in symbolic execution and fuzzing to enhance bug detection and code coverage for smart contracts.
- **Read-after-write dependency analysis:** the tool analyzes read-after-write dependencies to generate meaningful instruction sequences, thereby improving the relevance of the performed tests.
- **Environmental instrumentation:** ConFuzzius incorporates environmental information such as block timestamps and call return values to better simulate real-world scenarios.

- **Constraint solving:** the tool employs constraint solving to generate targeted and relevant inputs during fuzzing, increasing the effectiveness of vulnerability detection.

- **Advantages**

- **High code coverage:** ConFuzzius achieves a high level of code coverage, indicating thorough exploration of smart contract execution paths.
- **Enhanced vulnerability detection:** thanks to its advanced methods, ConFuzzius detects a significant number of vulnerabilities, bolstering smart contract security.
- **Efficient combination of techniques:** ConFuzzius' hybrid approach, combining symbolization and fuzzing, maximizes the benefits of each technique for optimal results.

- **Limitations**

- **Potential complexity:** the combined use of symbolization and fuzzing may make ConFuzzius more complex for novice users to understand and utilize.
- **Computational resource requirement:** the advanced methods used by ConFuzzius may require substantial computational resources, which could be a drawback in certain environments.

- **Future works**

- **Performance enhancement:** continuously optimizing bug detection performance and code coverage to make ConFuzzius even more effective.
- **Exploration of new techniques:** evaluating and integrating new testing and analysis techniques to enhance ConFuzzius' ability to detect a wider range of vulnerabilities.
- **Improved user interface:** developing a user-friendly interface to facilitate ConFuzzius' use by a broader range of users, including those who are not experts in smart contract security.

In conclusion, ConFuzzius stands as a powerful and innovative hybrid tool for evaluating and detecting vulnerabilities in smart contracts on blockchains. Thanks to its combination of symbolic execution and fuzzing techniques, it offers improved code coverage and enhanced vulnerability detection. Although it may have some drawbacks and require ongoing improvements, ConFuzzius represents a significant advancement in securing smart contracts and paves the way for future work to further strengthen its effectiveness and reliability.

J) Echidna

Echidna [17] is an advanced tool designed specifically for testing and fuzzing Ethereum smart contracts. As the security of smart contracts becomes increasingly critical, tools like Echidna play a vital role in identifying vulnerabilities and ensuring the robustness of smart contract code. In this overview, we'll explore the main features, advantages, disadvantages, and future developments of Echidna in enhancing smart contract security.

- **Main features**

- **Architecture:** Echidna's architecture is divided into two steps, namely, preprocessing and the fuzzing campaign.
- **Preprocessing:** Echidna uses Slither to analyze contracts and identify useful constants and functions that directly handle Ether (ETH).
- **Fuzzing campaign:** Echidna generates random transactions using the contract's ABI, important constants defined in the contract, and previously collected transaction sets.

- **Advantages**

- **Automatic property violation detection:** when a property violation is detected, Echidna automatically minimizes a counterexample to report the smallest sequence of transactions that triggers the failure.
- **Complexity minimization:** Echidna minimizes complexity by reporting the simplest sequence of transactions that causes a property violation.

- **Limitations**

- Specific property requirement: before testing a contract with Echidna, it is necessary to explicitly define specific properties in the contract or use Solidity's assertion to express the properties.

- **Future works**

- **Driver mode:** Echidna plans to add a driver mode similar to the swarm tool to leverage configuration diversity, including swarm testing, to fully exploit multicore machines.
- **Gas estimation:** Echidna intends to add gas estimation functionality to assess the maximum gas consumption of contracts.
- **Test corpus collection:** Echidna will work on the test corpus collection to enhance test generation.

These features, advantages, disadvantages, and future work demonstrate the power and ongoing improvement prospects of the Echidna tool for Ethereum smart contract fuzzing.

K) Harvey

Harvey [22] is a powerful greybox fuzzer designed to assess and detect bugs in smart contracts deployed on blockchains. With its advanced features, such as input prediction, demand-driven sequential fuzzing, and detection of various types of bugs, Harvey provides a robust approach to enhancing the security and reliability of smart contracts. Within this division, we will explore the key features, advantages, disadvantages, and future prospects of the Harvey tool.

- **Main features**

- **Input prediction:** Harvey uses input prediction to accelerate bug detection.
- **Demand-driven sequential fuzzing:** it focuses on generating transaction sequences to efficiently explore code paths.
- **Detection of various bugs:** Harvey can detect different types of bugs, including assertion violations and memory access errors.
- **Bug de-duplication:** uses a simple method to determine if two bugs are duplicates.
- **Utilization of advanced fuzzing techniques:** Harvey employs advanced techniques to improve code coverage and detect bugs faster.

- **Advantages**

- **Increased efficiency:** Harvey is more efficient than basic configurations in terms of bug detection and code coverage.
- **Fast bug detection:** input prediction speeds up bug detection.
- **Handling various bugs:** capable of detecting a range of bugs, including assertion violations and memory access errors.
- **Generation of transaction sequences:** the ability to generate transaction sequences helps uncover bugs requiring multiple transactions to reveal.

- **Limitations**

- **Potential complexity:** advanced configurations may be more complex to implement and manage than basic configurations.
- **Computational resource needs:** advanced configurations may require more computational resources due to the use of advanced techniques.

- **Future works**

- **Performance improvement:** continue to improve bug detection performance and code coverage.
- **Resource optimization:** explore ways to optimize the use of computational resources to make Harvey more efficient.
- **Extension of features:** add additional features such as detection of new types of bugs and support for more complex smart contracts.
- **Validation in other domains:** validate Harvey's effectiveness in other programming domains to ensure its generalizability.

In summary, Harvey is a powerful tool for bug detection in smart contracts, offering significant improvements in efficiency and code coverage. However, it may require additional effort to fully leverage, and there are opportunities for continuous improvements and functional extensions.

L) Reinforcement learning-guided fuzzer (RLF)

The RLF tool [21] represents a significant advancement in detecting vulnerabilities in smart contracts on blockchains. By combining reinforcement learning, symbolic execution, and fuzzing techniques, RLF aims to improve smart contract security by targeted identification of potential vulnerabilities. This study explores RLF's key features, advantages, limitations, and future improvement prospects.

- **Main features**

- RLF is a hybrid fuzzer guided by reinforcement learning, designed to detect vulnerabilities in smart contracts.
- It combines symbolic execution and fuzzing techniques to explore vulnerable execution paths and generate sequences of vulnerable transactions.
- The tool is designed to be data-dependent, allowing it to specifically target data-related vulnerabilities in smart contracts.

- **Advantages**

- RLF uses a reinforcement learning model to guide the fuzzing process, enabling efficient generation of vulnerable transaction sequences.
- It is capable of detecting a wide range of vulnerabilities, including Ether-Leaking and Suicidal Contracts, due to its ability to explore different execution paths.
- The tool can be effective even on real contracts deployed on the Ethereum blockchain, demonstrating its robustness in real-world environments.

- **Limitations**

- The main drawback of RLF is its dependency on training data and the diversity of vulnerabilities covered in that data. A limited training dataset can limit its ability to effectively detect rarer or less common vulnerabilities.
- Like any reinforcement learning-based tool, RLF requires a training and parameter adjustment phase to achieve optimal performance, which may require significant computational resources.

- **Future works**

- Improving the diversity and quality of training data to allow RLF to detect a wider range of vulnerabilities, including those that are less common or more complex.
- Optimizing reinforcement learning parameters and algorithms to improve the accuracy and efficiency of vulnerability detection.
- Extending RLF's compatibility to other blockchains and smart contract protocols for broader coverage and increased applicability.

- Integrating false positive filtering mechanisms to reduce the number of incorrect results and improve the overall accuracy of vulnerability detection.

In conclusion, the RLF tool offers an innovative and promising approach to enhancing smart contract security on blockchains. Its hybrid capabilities of reinforcement learning and fuzzing enable effective detection of critical vulnerabilities. While challenges remain, such as data training dependency and result accuracy, RLF represents a significant advancement in the field of smart contract security. Future work could focus on improving the accuracy and coverage of detected vulnerabilities, as well as optimizing the overall performance of the tool.

M) Gasfuzzer

GasFuzzer [19] stands out as a sophisticated tool, showcasing distinctive features that position it as a key player in the Ethereum smart contract security analysis domain.

• Main features

- **Gas-oriented exception vulnerability identification:** GasFuzzer is specifically designed to uncover gas-oriented exception security vulnerabilities in Ethereum smart contract binaries. It focuses on manipulating gas allowances to expose vulnerabilities related to gas consumption during code execution.
- **Two-phase approach:** GasFuzzer operates in two distinct phases. In the first phase, it employs a gas-greedy strategy to favor transactions with higher gas consumption, generating test transactions with varying gas consumptions. The second phase introduces a novel concept of fractional gas consumption coverage and a gas-leveling strategy, further enhancing the tool's ability to identify vulnerabilities.
- **Fractional gas consumption coverage:** GasFuzzer introduces a unique notion of fractional gas consumption coverage, providing a more nuanced understanding of gas-related vulnerabilities. This approach contributes to a more comprehensive assessment of smart contract security.
- **Real-world dataset evaluation:** GasFuzzer has been rigorously evaluated using a dataset comprising 3170 real-world smart contracts deployed on the public Ethereum blockchain. The evaluation demonstrates the tool's effectiveness in detecting Exceptions Disorder security vulnerabilities, surpassing the performance of previous black-box fuzzers.

• Advantages

- **Enhanced vulnerability detection:** GasFuzzer, particularly with its gas-greedy strategy, excels in detecting Exceptions Disorder vulnerabilities by favoring transactions with higher gas consumption. This leads to a more thorough identification of potential security issues in smart contracts.
- **Innovative gas-leveling strategy:** the introduction of a gas-leveling strategy in the second phase enhances GasFuzzer's capability to uncover vulnerabilities. By mutating the gas allowances of selected transactions, it provides a nuanced approach to fuzz testing, improving overall vulnerability detection.

• Limitations

- **Potential complexity:** GasFuzzer's effectiveness may be influenced by the complexity of smart contracts. The tool might face challenges in scenarios where smart contracts employ intricate structures or protection mechanisms, impacting its ability to identify vulnerabilities accurately.

• Challenges and future outlooks

- **Handling advanced protection mechanisms:** GasFuzzer may encounter challenges in dealing with advanced protection mechanisms implemented in smart contracts. Future developments could focus on enhancing the tool's capability to navigate through complex security features.
- **Scalability and performance:** as the Ethereum ecosystem evolves, ensuring GasFuzzer's scalability and performance becomes crucial. Future efforts may concentrate on optimizing the tool to handle an increasing number of complex smart contracts efficiently.
- **Integration of machine learning:** to stay ahead of emerging vulnerabilities, GasFuzzer could explore the integration of machine learning techniques. This would enable the tool to adapt and learn from new patterns, enhancing its ability to detect novel gas-oriented vulnerabilities in smart contracts.

In summary, GasFuzzer exhibits notable features, advantages, and innovative strategies in identifying gas-oriented exception vulnerabilities. However, ongoing efforts should address potential complexities and challenges while exploring advancements in handling evolving smart contract security landscapes.

N) xFuzz

xFuzz [11] is a cutting-edge machine learning-guided smart contract fuzzing framework designed to enhance the security of smart contracts.

• Main features

- **Machine learning guidance:** xFuzz employs machine learning models with innovative features such as word vectors and instructions. These models play a crucial role in guiding the smart contract fuzzing process.
- **Dynamic path filtering:** utilizing machine learning models, xFuzz intelligently filters likely benign program paths. This dynamic path filtering is more robust compared to traditional static tools, eliminating the need for manually-defined rules.
- **Cross-contract vulnerability detection:** xFuzz excels in detecting cross-contract vulnerabilities, exposing 15 vulnerabilities for the first time. This capability is crucial for identifying potential threats that span multiple smart contracts.
- **Efficiency in non-cross-contract vulnerability detection:** xFuzz demonstrates efficiency in detecting non-cross-contract vulnerabilities, boasting a detection rate twice as high as other fuzzing tools while consuming less than 20% of the time.

• Advantages

- **High precision and recall:** xFuzz achieved an impressive precision of 95.5% and a recall of 84.6%, surpassing the performance of other examined tools.
- **Number of detected vulnerabilities:** xFuzz successfully identified 209 real vulnerabilities between contracts, as well as 18 real vulnerabilities unrelated to contracts. These results highlight xFuzz's ability to cover a wide range of vulnerability scenarios.
- **Fuzzing time optimization:** thanks to its machine learning model, xFuzz significantly reduced fuzzing time on potentially benign contracts, achieving time savings of around 80%. This demonstrates its effectiveness and adaptability in diverse environments.
- **Path prioritization:** xFuzz's path prioritization strategy allowed it to focus efforts on the most relevant paths, leading to more efficient vulnerability detection.

tion. This efficient approach translates into more conclusive results.

- **Speed comparison with sFuzz:** xFuzz outperformed sFuzz in terms of speed, saving over 80% of research time while reporting a higher number of vulnerabilities in less than 20% of the time. This increased performance underscores the superior efficiency of xFuzz.
- **Discovery of previously unknown real vulnerabilities:** xFuzz successfully discovered vulnerabilities between contracts that were previously unknown in the real world.
- **Limitations and future improvements:** despite its successes, xFuzz has limitations, particularly in handling complex path conditions involving multiple variables and values. To overcome this challenge, the integration of a theorem prover like Z3 is suggested. This potential improvement would enable the tool to handle more complex path conditions, creating a hybrid fuzzing approach.

In summary, xFuzz's exceptional performance in terms of vulnerability detection and execution speed makes it a preferred tool for enhancing the security of smart contracts. Its results surpass those of its counterparts, and the suggested improvements reflect its ongoing commitment to innovation and efficiency.

3.2. Mutation testing

A) EtherFuzz

EtherFuzz [25] is a top-quality tool for identifying vulnerabilities in Ethereum contracts. It's effective, though it needs high-quality test data and complex setup. It's continually improving for better detection and easier use.

- **Main features**
 - **Preprocessing module:** consists of four steps, including compiling the smart contract, generating bytecode and ABI, generating test data, and instrumenting the bytecode.
 - **TOD vulnerability mutation fuzzing module:** involves three steps, namely mutating existing test data, monitoring fuzzing execution, and analyzing transaction order dependency (TOD) vulnerabilities.
- **Advantages**
 - **Efficient TOD vulnerability detection:** utilizes a custom TOD vulnerability mutation fuzzing algorithm to effectively detect vulnerabilities in smart contracts.
 - **Reduced false positive rate:** compared to other vulnerability detection tools, EtherFuzz manages to significantly reduce the false positive rate, enhancing result reliability.
 - **Reduced detection costs:** offers lower detection costs in terms of time and memory compared to other tools, enabling quicker analysis of smart contracts.
- **Limitations**
 - **Dependency on initial test data:** the quality of detection results heavily depends on the quality of the initial test data provided.
 - **Configuration complexity:** the initial setup of the tool can be complex and require in-depth expertise in smart contract security.
- **Future works**
 - **Improvement of detection algorithm:** further research can be conducted to enhance the vulnerability detection algorithm, including introducing new fuzzing techniques or optimizing existing methods.

- **Expansion of vulnerability support:** EtherFuzz could be extended to detect a wider range of vulnerabilities in smart contracts, including those requiring more complex interaction with the Ethereum network.
- **Simplification of use:** efforts to simplify the installation, configuration, and use of EtherFuzz could make the tool more accessible to a broader range of developers and security researchers.

In summary, EtherFuzz shows promising potential to enhance the security of smart contracts by effectively detecting vulnerabilities. Despite its advantages, improvements are needed to maximize its utility and ease of use. Ongoing research could help expand its scope and reduce its limitations, thereby strengthening trust in the blockchain application ecosystem.

B) SuMo

SuMo (symbolic mutator for solidity) [26] is a mutation testing tool specifically designed for Solidity smart contracts. Its main characteristics, advantages, limitations, and potential future work are described below:

- **Main features**

- **Mutation operators:** SuMo provides a set of mutation operators that can be applied to Solidity smart contracts. These operators simulate faults or introduce variations in the code to evaluate the robustness of the test suite.
- **Mutation score calculation:** SuMo calculates the mutation score based on the number of generated mutants, stillborn mutants (mutations that cannot be executed due to syntax or semantic errors), and equivalent mutants (mutations that do not affect the behavior of the contract).
- **Test suite improvement:** SuMo can help improve test suites by identifying areas where tests are weak or missing. By analyzing surviving mutants, developers can create additional test cases to improve code coverage and fault detection.
- **Manual inspection:** after mutation testing, SuMo allows for manual inspection to identify and remove equivalent mutants. This step ensures that the mutation score accurately reflects the effectiveness of the test suite.
- **Integration with Solidity tools:** SuMo integrates with specific Solidity tools and frameworks, making it easier to incorporate mutation testing into the development workflow for Ethereum smart contracts.

- **Advantages**

- **Fault detection:** SuMo can effectively detect weaknesses in test suites by generating mutants that simulate common programming errors. This helps identify areas of the code that are not adequately tested.
- **Test suite enhancement:** by analyzing surviving mutants, developers can improve their test suites, leading to better code quality and fewer vulnerabilities in smart contracts.
- **Integration:** SuMo's integration with existing Solidity tools and frameworks makes it convenient for developers to adopt mutation testing as part of their development process.

- **Limitations**

- **Manual inspection:** the manual inspection step after mutation testing can be time-consuming and may require domain expertise to identify and remove equivalent mutants accurately.
- **Limited mutation operators:** while SuMo's set of mutation operators is comprehensive, it may not

cover all possible mutation scenarios, leading to potential blind spots in fault detection.

- **Performance overhead:** generating and analyzing mutants can introduce some performance overhead, especially for large and complex smart contracts, which may impact development speed.
- **Future works**
 - **Expansion of mutation operators:** continuously expanding the set of mutation operators to cover more edge cases and scenarios can enhance SuMo's fault detection capabilities.
 - **Automation:** streamlining the manual inspection process and potentially automating the identification and removal of equivalent mutants can improve the efficiency of mutation testing.
 - **Scalability:** optimizing SuMo's performance for scalability, especially with larger codebases and extensive test suites, will make it more practical for real-world applications.
 - **Integration with CI/CD pipelines:** integrating SuMo seamlessly into continuous integration and continuous deployment (CI/CD) pipelines can promote its adoption and usage among developers.
 - **Community collaboration:** encouraging collaboration and feedback from the Solidity development community can lead to iterative improvements and feature enhancements for SuMo.

In short, SuMo enhances Solidity smart contract testing by identifying faults and suggesting improvements. It needs further development to automate tasks and improve scalability. Collaborative efforts can make SuMo a powerful tool for ensuring smart contract reliability and security.

3.3. Symbolic execution

A) SMARTEST

SMARTEST [27] is a promising tool that leverages language models to detect vulnerabilities. This analysis explores its features, advantages, limitations, and future prospects.

- **Main features**
 - **Use of language models:** SMARTEST utilizes language models to guide symbolic execution, significantly enhancing the effectiveness of vulnerability detection in smart contracts.
 - **Comprehensive transaction sequence analysis:** the tool explores various possible transaction sequences to identify vulnerabilities, covering a wide range of potential scenarios.
 - **Automatic vulnerability validation:** SMARTEST has an automatic validator that confirms the presence of detected vulnerabilities, ensuring precise and reliable results.
 - **Support for multiple vulnerability types:** the tool is capable of detecting several common types of vulnerabilities in smart contracts, including integer overflows, assertion violations, Ether leaks, suicidal transactions, and ERC20 standard violations.
 - **Fast execution time:** thanks to the use of language models and an efficient symbolic execution methodology, SMARTEST manages to detect vulnerabilities in a relatively short period, enabling developers to react promptly.
- **Advantages**
 - **Improved efficiency:** SMARTEST surpasses several existing tools in terms of smart contract vulnerability

detection, thereby offering a more robust solution to enhance blockchain application security.

- **Automatic validation:** automatic validation of detected vulnerabilities ensures reliable results and saves developers the time required to manually verify each result.
- **Speed of execution:** the tool provides fast execution times due to the use of language models and efficient symbolic execution techniques, enabling quick analysis of smart contracts.
- **Limitations**
 - **Dependency on language models:** SMARTEST's effectiveness heavily relies on the quality of the language models used, which could limit its ability to detect vulnerabilities in cases where the models are not suitable.
 - **Need for training data:** SMARTEST requires large and diverse training datasets to function optimally, which can sometimes be challenging to obtain, especially for less common vulnerabilities.
 - **Configuration complexity:** configuring SMARTEST and associated tools can be complex and requires considerable expertise in the field of smart contract security.
- **Future works**
 - **Improvement of language models:** further research can be undertaken to enhance the language models used by SMARTEST, which could further improve its performance and ability to detect new vulnerabilities.
 - **Expansion of vulnerability support:** SMARTEST could be expanded to detect a broader range of vulnerabilities in smart contracts, including those requiring interaction between multiple contracts.
 - **Simplification of configuration:** efforts to simplify the configuration and usage of SMARTEST could make the tool more accessible to a wider range of developers and security researchers.

In summary, SMARTEST represents a breakthrough in smart contract security through its use of language models. Despite challenges such as model dependency and the need for training data, its future potential is promising.

B) WebAssembly network analysis (WANA)

The WANA [28] symbolic smart contract analysis framework represents a significant advancement in securing blockchain ecosystems, particularly on EOSIO and Ethereum platforms. Addressing the growing demand for automated vulnerability analysis methods, WANA provides a sophisticated solution to identify and address potential security flaws in smart contracts, thereby reducing the risks of malicious exploitation.

- **Architecture and features**

WANA distinguishes itself through its use of symbolic analysis of WebAssembly (Wasm) bytecode, offering unparalleled depth of analysis. Its symbolic execution engine traverses code execution paths with symbolic inputs, allowing for exhaustive evaluation of potential smart contract behaviors. By collecting execution information and using it to detect specific vulnerabilities, such as fake EOS transfers or forged transfer notifications, WANA provides detailed reports enabling developers to effectively secure their contracts.
- **Advantages**

One of the main advantages of WANA is its ability to automate the vulnerability analysis process, thereby reducing the time required to identify potential risks. This automation also enables thorough and reproducible analysis of smart contracts, ensuring comprehensive coverage of possible security flaws. Furthermore, WANA's compatibility with leading

blockchain platforms such as EOSIO and Ethereum makes it a versatile and widely applicable tool in various development environments.

- **Limitations**

While powerful, WANA is not without limitations. Complex symbolic analyses may require significant computational resources, which can extend execution times and limit the tool's scalability. Additionally, the possibility of false positives or false negatives necessitates additional manual verification to ensure result accuracy. Despite these challenges, the team behind WANA remains committed to continually improving and refining the framework to meet the evolving needs of the blockchain community and ensure the long-term security of smart contracts.

- **Future works**

- **Extension to web application vulnerability detection:** with the widespread adoption of Wasm in web browsers, there is a growing need for vulnerability detection tools for web applications. Extending WANA to support the detection of vulnerabilities in web applications would address this demand, providing developers with a comprehensive solution for securing their web-based systems.
- **Enhanced support for external function analysis:** improving WANA's ability to handle the symbolic analysis of external functions will be crucial for accurately identifying vulnerabilities in smart contracts. By refining its techniques for analyzing interactions with external systems and APIs, WANA can offer more precise vulnerability detection across a wider range of scenarios.
- **Integration with development environments:** integrating WANA with popular development environments and tools used by smart contract and web application developers would streamline the vulnerability detection process. By offering seamless integration with integrated development environments and code editors, developers can easily incorporate vulnerability checks into their development workflows, enhancing the security posture of their projects.
- **Expansion of platform support:** while WANA currently focuses on EOSIO and Ethereum smart contracts, there is potential for extending its support to other blockchain platforms and ecosystems. By adapting its capabilities to different platforms and their specific characteristics, WANA can cater to a broader audience of developers and stakeholders within the blockchain community.
- **Community engagement and feedback:** engaging with the community of developers, security researchers, and blockchain enthusiasts will be vital for the continued improvement and refinement of WANA. Soliciting feedback, addressing user concerns, and incorporating feature requests will ensure that WANA remains relevant and effective in addressing the evolving challenges of smart contract and web application security.

By pursuing these future prospects, WANA can further solidify its position as a leading cross-platform vulnerability detection framework, empowering developers to build more secure and resilient blockchain-based systems and web applications.

C) Reentrancy analyzer (RA)

RA [29] is a state-of-the-art tool tailored for Ethereum smart contracts. It focuses on detecting and mitigating reentrancy attacks, a major security concern in blockchain development. By leveraging symbolic execution and satisfiability modulo theories (SMT) solving

techniques, RA enhances contract security and contributes to a safer ecosystem.

- **Main features**

- **Static analysis combined with SMT solver usage:** RA uses a static analysis approach that combines symbolic execution with an SMT solver. This allows RA to symbolically emulate reentrancy attacks and verify vulnerabilities without requiring the actual execution of contracts.
- **Inter-contract analysis:** unlike some previous methods, RA is capable of analyzing interactions between contracts, including calls to external contracts. This significantly improves the coverage of reentrancy attack analysis.
- **Precise vulnerability verification:** RA aims to provide precise vulnerability verification, without false positives or false negatives. It can detect reentrancy attacks without requiring prior knowledge of the attack patterns.
- **Use of EVM bytecode as input:** RA can be used to analyze Ethereum contracts using their EVM bytecode as input, making it independent of the specific compiler used to generate the bytecode.

- **Advantages**

- **Accuracy:** RA offers high accuracy in detecting reentrancy vulnerabilities, making it a reliable tool for smart contract developers and auditors.
- **Extended coverage:** by enabling inter-contract analysis, RA improves the coverage of reentrancy attack analysis, which is crucial for Ethereum contract security.
- **Ease of use:** RA users do not need to have in-depth knowledge of reentrancy attacks or specific attack patterns. The tool can be used by simply providing the bytecode of the target contracts.
- **Reasonable performance:** although analyzing some contracts may take time due to the complexity of inter-contract interactions, RA generally offers reasonable performance for static analysis of Ethereum contracts.

- **Limitations**

- **Non consideration of gas consumption:** currently, RA does not consider gas management in contracts, which could lead to limitations in the accurate analysis of certain contracts that rely on gas consumption constraints.
- **Limitation on the number of interconnected contracts:** RA is limited in its ability to effectively analyze cases where multiple contracts are tightly linked and interdependent.
- **Absence of symbolic values in bytecode:** RA does not support cases where the created contracts are different for each execution, which could limit its ability to detect certain vulnerabilities.

- **Future works**

- **Improvement of inter-contract interaction support:** a future improvement of RA could include better support for inter-contract interactions, including analysis of DELEGATECALL or CALLCODE instructions.
- **Integration of gas management:** integrating gas management into the analysis could improve the accuracy of RA and allow it to better understand the behavior of Ethereum contracts under real conditions.
- **Support for symbolic values in bytecode:** by supporting symbolic values in bytecode, RA could extend

its ability to detect a wider range of potential vulnerabilities in Ethereum contracts.

Overall, RA represents a significant advancement in the static analysis of Ethereum contracts and offers considerable potential for enhancing the security of smart contracts on the Ethereum platform.

D) SAILFISH

SAILFISH [30] is a set of tools for analyzing the security of smart contracts on the Ethereum blockchain. It consists of two main modules: EXPLORER and REFINER. EXPLORER creates a storage dependency graph (SDG) to detect vulnerabilities, while REFINER performs value analysis to enhance contract security.

• Main features

- SAILFISH consists of two main modules: EXPLORER and REFINER. The EXPLORER module constructs an SDG from the source of a contract.
- The SDG overestimates read-write accesses to storage variables along all possible execution paths. State inconsistency vulnerabilities are modeled as graph queries on the SDG.
- If the query result is an empty set, the contract is certainly not vulnerable; otherwise, a counter-example is generated. The REFINER module validates counterexamples returned by the EXPLORER.
- It performs contract-wide value summary analysis to mitigate potential issues arising from path conditions involving public methods other than those initially examined.

• Advantages

- **Effective vulnerability detection:** SAILFISH effectively detects vulnerabilities related to reentrancy and Ether flow alteration attacks (TOD) in Ethereum smart contracts.
- **Scalability:** compared to other tools, SAILFISH offers better scalability, allowing it to quickly analyze a large number of contracts, even those of significant size.
- **Precision:** SAILFISH maintains a low false positive rate, ensuring high precision in vulnerability detection.
- **Ability to detect zero-day vulnerabilities:** SAILFISH is capable of discovering zero-day vulnerabilities, thus providing advanced protection against emerging attacks.

• Limitations

- **Source code dependency:** SAILFISH relies on access to the contract source code, which may limit its applicability to contracts with accessible source code.
- **Potential unsoundness:** although SAILFISH is designed to detect various vulnerabilities, it may not be entirely foolproof due to the contract complexity and limitations of static analysis techniques.
- **Need for updates:** like any security tool, SAILFISH requires regular updates to remain effective against new threats and vulnerabilities emerging in the smart contract ecosystem.

• Future works

- **Improving precision:** SAILFISH could benefit from improving its analysis techniques to further reduce false positives and false negatives.
- **Expanding support:** expanding SAILFISH's support to include other types of vulnerabilities and programming languages could increase its utility and relevance in a broader context.
- **Integration of machine learning:** integrating machine learning techniques could enable SAILFISH to more effectively identify vulnerability patterns in

smart contracts, enhancing its ability to detect emerging vulnerabilities.

In conclusion, SAILFISH offers an innovative approach to analyzing the security of smart contracts on the Ethereum blockchain. It provides a comprehensive solution for detecting and enhancing contract security, which is essential in an environment where trust and reliability are paramount.

E) teEther

TeEther [31] is a tool designed for the automatic identification of vulnerabilities and the generation of exploits in Ethereum smart contracts. By analyzing 38,757 contracts, it successfully generated 1,564 functional exploits, highlighting significant security flaws in the implementation of smart contracts.

• Key features

- **Automatic vulnerability detection:** teEther is designed to automatically identify vulnerabilities in Ethereum smart contracts by analyzing their binary bytecode, enabling efficient evaluation of numerous contracts.
- **Exploit generation:** the tool not only detects vulnerabilities but also generates exploits for them, allowing attackers to exploit these vulnerabilities without manual intervention.
- **Large-scale analysis capability:** teEther has been tested on a vast dataset of 38,757 unique Ethereum contracts, successfully identifying functional exploits for 815 of them, demonstrating its effectiveness in real-world applications.

• Advantages

- **Efficiency in security assessments:** automating vulnerability detection and exploit generation significantly reduces the time and effort required for security evaluations, making it easier for researchers to identify weaknesses.
- **Wide applicability:** the tool's generic definition of vulnerabilities makes it applicable to a wide range of contracts, enhancing its usefulness in the field of smart contract security.
- **Insights into vulnerability prevalence:** by analyzing a large number of contracts, teEther provides valuable insights into the prevalence of vulnerabilities within the Ethereum ecosystem, highlighting critical security issues.

• Limitations

- **Potential for misuse:** while teEther can assist security researchers, it also poses risks as malicious actors could exploit it to target vulnerable contracts, resulting in financial losses.
- **Reliance on binary analysis:** the use of binary bytecode may limit the tool's effectiveness in identifying certain types of vulnerabilities that are more apparent in high-level code.

• Future work

- **Developing enhanced protection mechanisms:** the authors suggested that as exploiting smart contracts becomes more lucrative, there is a growing need to develop better protection mechanisms to safeguard against identified vulnerabilities.
- **Expanding vulnerability analysis:** future research could focus on broadening the analysis to include more complex contracts, different blockchain platforms, and improving the tool's ability to detect a wider variety of vulnerabilities.
- **Addressing critical vulnerabilities:** the paper emphasizes the importance of continued efforts to address the critical vulnerabilities identified in their

analysis, as the issue is more significant than previously thought.

TeEther is a powerful tool for identifying and exploiting vulnerabilities in Ethereum smart contracts, offering notable advantages and limitations. It also underscores the need for ongoing research and development in smart contract security.

F) DefectChecker

DefectChecker [32] is a defect detection tool for Ethereum smart contracts, primarily based on bytecode analysis. Here is a summary of the main points:

• Main features

- **Modular architecture:** the tool comprises four modules: bytecode input, control flow graph generation, feature detection (such as money calls and payable functions), and defect identification.
- **Bytecode analysis:** utilizes the Geth API to disassemble bytecode and perform security checks on deployed contracts on the blockchain, even without the source code.
- **Defect detection:** identifies vulnerabilities such as reentrancy, DoS by external influence, unchecked external calls, and other security risks.
- **Symbolic execution:** executes bytecode instruction by instruction with an SMT solver to analyze conditions and jumps within the code.
- **Detected features:** the tool identifies specific features like “Money Calls,” infinite loops, and payable functions that may be vulnerable.
- **Private bytecode support:** can analyze contracts without open-source code, relying only on bytecode.

• Advantages of DefectChecker

- **In-depth bytecode analysis:** DefectChecker can analyze Ethereum smart contract bytecode, even without the source code, allowing for security verification of deployed contracts on the blockchain.
- **Detection of specific vulnerabilities:** the tool can identify common vulnerabilities such as reentrancy, infinite loops, and unsecured external calls, enabling rapid identification of security risks.
- **Modular architecture:** its modular design allows easy adjustments or expansions to integrate new checks or adapt them for specific use cases.
- **Symbolic execution:** using symbolic execution with an SMT solver enables more precise analysis of execution paths and control conditions, improving test coverage.
- **Support for private bytecode:** the tool can analyze contracts without available source code, providing security coverage for private or closed contracts.

• Limitations

- **Dependence on bytecode:** the tool can only analyze contracts with available bytecode. If the bytecode is modified or optimized in a way that makes it harder to analyze, it may limit its effectiveness.
- **Lack of support for certain defect types:** while effective in detecting some vulnerabilities, DefectChecker may not cover all possible types of vulnerabilities, particularly those requiring deeper analysis of complex contract interactions.
- **Limited dynamic security analysis:** DefectChecker primarily relies on static bytecode analysis. Dynamic testing or executing real attacks in simulated environments may not be fully covered.
- **Integration complexity:** while the tool is modular, integrating it into complex workflows or adapting it

for specific test environments may require technical effort.

- **Performance on large contracts:** symbolic analysis and step-by-step execution can be slow for very complex or large contracts, making the tool less suitable for large codebases.

• Future work

- **Integration with SMT solvers:** a key direction for future work is to explore combining DefectChecker’s current method with an SMT solver. This integration could help balance efficiency and accuracy by reducing false positives and negatives in defect detection (e.g., NC, DuEI). Although an SMT solver might improve accuracy, it could also increase time consumption for contract analysis, so a balance needs to be achieved.
- **Identifying code patterns causing errors:** researchers could investigate specific code patterns that lead to errors in DefectChecker, such as those causing false positives or negatives. For instance, DefectChecker assumes all paths are reachable, but certain conditional expressions may always evaluate to false, leading to false positives in loop detection. To address this, SMT solvers can be used to check conditional expressions, especially in loop-related blocks, to improve accuracy in loop detection.
- **Enhancing efficiency:** another direction for future work is to optimize the tool for faster analysis without sacrificing accuracy, especially when handling large volumes of contracts. This could involve improving the internal processes for symbolic execution or streamlining verification stages.
- **Expansion to detect more contract defects:** future iterations of DefectChecker could aim to detect more types of smart contract defects beyond the eight currently identified, broadening the tool’s applicability.
- **Broader application:** DefectChecker could be expanded to support contract analysis on other blockchain platforms beyond Ethereum, extending its utility in the blockchain development community.

3.4. Test metamorphic

MT is not a specific tool but rather a methodological approach used in the field of software testing [33]. This method relies on identifying and leveraging MRs, which are fundamental properties of the software that enable the generation of additional test cases or the validation of results in various contexts.

Although this approach is highly effective, it is typically implemented through third-party tools or integrated into more comprehensive testing frameworks. Therefore, this article does not focus on MT itself, as our primary objective is to present and analyze specific tools. Nonetheless, we acknowledge the value of this methodology and its significant potential to enhance testing strategies.

4. RQ3: What are the most common vulnerabilities identified in Ethereum smart contracts?

Smart contracts, central to DeFi and blockchain applications, bring innovative solutions but also pose significant risks. Vulnerabilities in these contracts can lead to considerable financial losses, as attackers exploit weaknesses to divert funds or disrupt services. Recent incidents involving issues like reentrancy vulnerabilities and integer overflows highlight the scale of risk when robust security is lacking. This underscores the importance of a structured approach to better understand and mitigate these vulnerabilities.

Classifying vulnerabilities plays a key role in strengthening the security of smart contracts by providing developers, auditors, and researchers with a clear reference framework. This classification helps to target the most dangerous flaws, develop more effective detection tools, and enhance the resilience of decentralized platforms.

This presentation aims to explore two approaches to vulnerability classification in smart contracts: exploitable vulnerabilities and those cataloged by the open smart contract vulnerabilities (OpenSCV) taxonomy. The main objectives are as follows:

- **Examine exploitable vulnerabilities:** this category covers weaknesses that are actively exploitable by attackers and often undetected by current tools, representing the most critical threats to smart contract security.
- **Analyze the OpenSCV framework:** widely recognized in the industry, this taxonomy provides an exhaustive classification of common vulnerabilities, though it may lack certain emerging or context-specific flaws recently discovered.

This comparison will provide insight into how these classifications contribute to better smart contract security and help formulate defensive strategies suited to today's blockchain challenges.

4.1. Exploitable vulnerabilities

Exploitable vulnerabilities [37] in smart contracts are weaknesses that allow attackers to manipulate or compromise the functioning of these contracts for malicious purposes. These flaws, often subtle and complex, enable critical attacks that jeopardize funds and compromise the reliability of dApps. Although numerous audit and security tools have been developed to detect these vulnerabilities, many of them evade existing detection mechanisms, making identification challenging and prevention essential.

These vulnerabilities are termed "exploitable" because they provide attackers with tangible attack vectors that can be leveraged to gain unauthorized privileges or benefits. Unlike minor errors or inconsequential bugs, an exploitable vulnerability presents a real and immediate breach in contract security, which can be used to cause financial damage, distort transactions, or alter contract behavior.

4.1.1. Methodology for classifying exploitable bugs in smart contracts

The article "Demystifying Exploitable Bugs in Smart Contracts" is based on a systematic empirical analysis of 516 unique vulnerabilities found in deployed smart contracts between 2021 and 2022. This methodology comprises several key steps, each contributing to an in-depth understanding of exploitable bugs.

- **Data collection:** the authors gathered information on real-world vulnerabilities, focusing on incidents that led to significant financial losses. This comprehensive dataset, drawn from multiple sources, offers a broad perspective on exploitable vulnerabilities in practical scenarios and their financial impacts.
- **Exploitability analysis:** each vulnerability was assessed to determine its potential to be exploited by attackers. The authors specifically focused on weaknesses that escape conventional analysis tools, highlighting gaps in current auditing practices. This step was crucial in identifying the most dangerous vulnerabilities for smart contracts.
- **Categorization by type:** the identified vulnerabilities were grouped into seven distinct types based on their characteristics and exploitability. This classification involves analyzing the root causes of each vulnerability, their distribution across various contracts, and the challenges associated with their auditing. The goal of this classification is to facilitate understanding and management of these vulnerabilities.
- **Impact assessment and mitigation strategies:** for each type of vulnerability, the authors examined the potential consequences of exploitation and proposed remediation strategies. This analysis includes possible financial impacts and provides developers with best practices to mitigate risks in future smart contract deployments.

- **Abstraction of bug patterns:** whenever possible, the authors generalized each type of vulnerability into bug patterns. These abstractions enable the identification of similar weaknesses in other contracts and support the automation of future detection processes, allowing developers and security auditors to proactively identify and fix these vulnerabilities.

This empirical analysis and structured classification framework provide a valuable tool for understanding and managing exploitable vulnerabilities in smart contracts, contributing to improved security in the blockchain ecosystem.

4.1.2. Types of exploitable vulnerabilities identified

4.1.2.1. Price oracle manipulation Description: this vulnerability occurs when an attacker manages to influence the price values provided by oracles for personal gain by manipulating the information transmitted to smart contracts. This can lead to economic imbalances and significant losses for users. Attack mechanism: the attacker manipulates the price of assets by exploiting the dependency of contracts on external oracles. This manipulation allows altering financial transactions in the attacker's favor, often at the expense of other participants.

4.1.2.2. Privilege escalation Description: this type of vulnerability happens when an unauthorized user gains access to normally restricted functions or privileges, enabling unintended actions such as accessing funds or modifying sensitive data. Attack Mechanism: The attacker exploits flaws in permission management or gains indirect access to certain functions, achieving a higher level of control than what should be permitted. This flaw can be used to divert funds or take control of sensitive contract functions.

4.1.2.3. Atomicity violations

- Description: this atomicity issue occurs when sequences of transactions, which are supposed to be executed as an indivisible unit, are interrupted, leading to inconsistencies in the contract's state.
- Attack mechanism: by inserting transactions during critical operations, an attacker can disrupt the logical sequence of operations and compromise the integrity of data or funds, resulting in inconsistencies or losses.

4.1.2.4. ID uniqueness violations

- Description: these violations involve errors in generating or managing unique identifiers, which can lead to traceability issues or data collisions.
- Attack mechanism: by exploiting weaknesses in the generation or verification of identifier uniqueness, an attacker can create conflicts in identification, potentially allowing unauthorized access or modification of resources tied to those identifiers.

4.1.2.5. Inconsistent state updates

- Description: this type of vulnerability arises when state updates are not properly synchronized, creating contradictions in the information stored by the contract.
- Attack mechanism: by disrupting the logical flow of state update operations, an attacker can introduce inconsistencies into the data. This can lead to unexpected behaviors, such as incorrect fund transfers or execution errors.

4.1.2.6. Contract implementation specific bugs

- **Description:** these vulnerabilities result from errors or flaws specific to the internal logic of a contract, inherent to the individual implementation of a contract.
- **Attack mechanism:** attackers exploit logical weaknesses or coding errors specific to a contract's implementation. These errors may include flaws in calculating critical values, handling certain transaction types, or missing security checks.

4.1.2.7. Erroneous accounting

- **Description:** accounting errors occur when the contract fails to properly record financial information or accounting states, which can lead to payment errors or improper allocation of funds.
- **Attack mechanism:** by manipulating or exploiting errors in the contract's financial records, an attacker can alter the distribution of funds or create accounting inconsistencies, leading to financial loss for users or the contract owner.

These seven types of vulnerabilities are critical because they provide direct exploitation opportunities for attackers, especially if they are not detected before contract deployment. Each of these vulnerabilities has the potential to cause significant financial loss or disruptions in the functioning of smart contract platforms.

4.2. The OpenSCV taxonomy

4.2.1. Analysis methodology

Smart contracts form the cornerstone of blockchain systems, facilitating transactions among entities. Despite their pivotal role, smart contracts, like conventional software, are susceptible to defects, including critical security vulnerabilities.

A) Addressing vulnerabilities: research initiatives and classification frameworks

Various research endeavors aim to detect and mitigate vulnerabilities in smart contracts, leading to the development of specialized vulnerability detection tools. While classification frameworks like DASP and SWC have emerged to categorize these vulnerabilities, their relevance has diminished over time due to evolving threats and advancements in smart contract technology.

B) Introducing OpenSCV: a novel taxonomic approach

To address the shortcomings of existing classification schemes, OpenSCV [38], a novel hierarchical taxonomy of smart contract vulnerabilities, has been devised. Unlike its predecessors, OpenSCV remains open to community contributions and is designed to evolve alongside emerging threats and technological advancements.

C) Constructing OpenSCV: methodology and rigor

The development of OpenSCV involved a meticulous analysis of existing vulnerability classifications and an extensive examination of 49 research works on vulnerability detection. The taxonomy comprises 76 vulnerabilities meticulously organized into a coherent hierarchical structure, ensuring comprehensive coverage of potential risks.

D) Flexibility and practical utility

OpenSCV stands out for its adaptability and practical utility. With a dedicated support infrastructure on GitHub, it welcomes continuous feedback and updates from the community. Moreover, each vulnerability entry is accompanied by a concrete code example, enhancing clarity and facilitating understanding.

E) Advancements and significance

OpenSCV represents a significant leap forward in smart contract vulnerability research, offering a dynamic and comprehensive classification framework. By addressing the limitations of outdated schemes and embracing the ever-evolving nature of smart contract technology, OpenSCV serves as an indispensable resource for researchers, developers, and security professionals alike, empowering them to better understand, prevent, and mitigate smart contract vulnerabilities.

After detailing the analysis methodology used to assess the smart contract vulnerabilities and presenting the OpenSCV taxonomy along with its benefits, we showcase these concepts through Table 2. This table lists various security vulnerabilities of smart contracts, providing details on each vulnerability category, as well as relevant research works that have studied or addressed these specific issues.

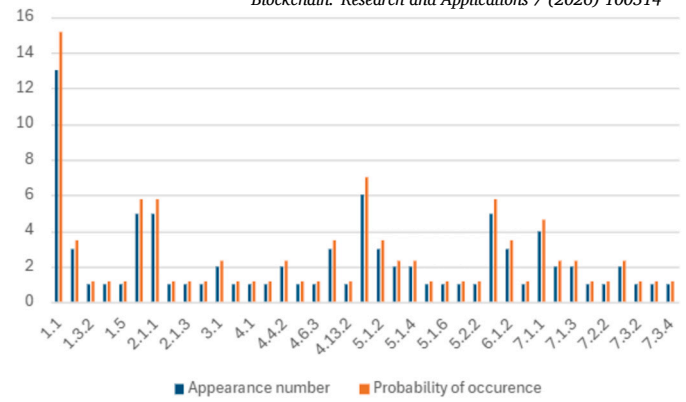


Fig. 3. Number of research works addressing each open smart contract vulnerabilities (OpenSCV) taxonomy.

4.3. Results of vulnerability analysis

After compiling vulnerabilities based on the OpenSCV taxonomy and associating each vulnerability with relevant research works, the results were summarized in Fig. 3, which is a diagram illustrating the number of works addressing each vulnerability category. This transition highlights the systematic organization of vulnerabilities in the table and transforms into a visual representation quantifying the research efforts dedicated to each vulnerability. The diagram serves as a visual aid to emphasize the varying degrees of attention and research focus given to different smart contract vulnerabilities, as identified by the OpenSCV taxonomy.

Fig. 3 provides an overview focusing on the identification and analysis of the most common vulnerabilities in smart contracts, classified according to the OpenSCV taxonomy. By evaluating the number of research papers dedicated to each vulnerability category between 2020 and 2023, we aim to determine the most concerning areas in smart contract security. Additionally, we examine less studied vulnerabilities, shedding light on security aspects deserving increased attention.

This analysis aims to provide valuable insights to guide future research and development efforts in smart contract security. By better understanding the most common vulnerabilities and identifying under-researched areas, we aim to enhance security and trust within the smart contract ecosystem.

After a thorough analysis of the diagram, we can identify the main vulnerabilities that have been most studied.

4.3.1. Reentrancy (1.1)

Calling external contracts introduces a significant risk as the control flow can be hacked. This vulnerability, known as a reentrancy attack or recursive call attack, occurs when a malicious contract initiates a callback to the calling contract before the initial function invocation is finished. Consequently, this can result in unintended and undesirable interactions between multiple invocations of the function [39]. Fig. 4 presents a scenario of two contracts interacting with each other, contract A and contract B [40].

In this situation, contract A invokes a function called B() in contract B. However, it is important to note that contract B possesses a vulnerability known as reentrancy. The reentrancy vulnerability occurs when a contract B, upon being called by contract A, permits contract A to recursively invoke function B() before completing its initial execution. This creates a loop where function B() can be repeatedly called before its execution is fully completed. This situation can result in undesirable outcomes. For instance, let's consider a scenario where function B() carries out a funds transfer operation. If the reentrancy vulnerability is not adequately addressed, an attacker can exploit the callback loop to execute multiple fund transfers before contract B has an opportunity to update the balance or implement necessary security measures [41].

Table 2

Vulnerabilities addressed by security testing tools based on the OpenSCV taxonomy.

Taxonomy OpenSCV			xFuzz	IR-Fuzz	ItyFuzz	SMARTIAN	DEPOSafe	Funfuzz	Etherolic	GasFuzzer	MagicMiroir	EF/CF	EtherFuzz	DefectChecker	SMARTEST	WANA	RA	SALESH	Metamorphic testing	teEther
1. Unsafe External Calls	1.1	1.1.1	✓	✓	✓	✓		✓	✓	✓	✓	✓		✓			✓	✓	✓	
		1.1.2	✓	✓	✓	✓		✓	✓	✓	✓	✓		✓			✓	✓	✓	
	1.2	1.2.1																		
		1.2.2																		
	1.3	1.3.1		✓										✓						✓
		1.3.2												✓						
		1.3.3																		
2. Mishandled Events	1.4	1.4.1						✓												
		1.4.2																		
		1.4.3																		
	1.5	1.5.1																		
		1.5.2																		
		1.5.3																		
	1.6	1.6.1	✓	✓						✓						✓				✓
3. Gas Depletion	2.1	2.1.1				✓			✓	✓								✓	✓	
		2.1.2												✓						
		2.1.3												✓						
	2.2	2.2.1					✓													
		2.2.2																		
	3.1	3.1.1								✓									✓	
		3.1.2																		
		3.1.3																		
4. Bad Programming Practices & Language Weakness	4.2	4.2.1							✓											
		4.2.2																		
		4.2.3																		
	4.3	4.3.1																		
		4.3.2				✓														
		4.3.3																		
	4.4	4.4.1																		
		4.4.2				✓									✓					
		4.4.3																		
	4.6	4.6.1													✓					
		4.6.2																		
		4.6.3													✓					
	4.7	4.7.1		✓					✓	✓										
		4.7.2																		
		4.7.3																		
	4.8	4.8.1																		
		4.8.2																		
		4.8.3																		
	4.10	4.10.1																		
		4.10.2																		
		4.10.3																		
	4.12	4.12.1																		
		4.12.2																		
		4.12.3																		
	4.13	4.13.1																		
		4.13.2				✓														
	4.14	4.14.1																		
		4.14.2																		
		4.14.3																		
5. Incorrect Control Flow	5.1	5.1.1		✓		✓		✓	✓	✓			✓			✓		✓		
		5.1.2																		
		5.1.3		✓										✓						
		5.1.4											✓	✓						
		5.1.5																		
		5.1.6											✓							
		5.1.7																		
	5.2	5.2.1							✓											
		5.2.2									✓									

(continued on next page)

Table 2 (continued)

Taxonomy OpenSCV			xFuzz	IR-Fuzz	ItyFuzz	SMARTAN	DEPOSafe	Funfuzz	Etherolic	GasFuzzer	MagicMiroir	EF/CF	EtherFuzz	DefectChecker	SMARTTEST	WANA	RA	SAILFISH	Metamorphic testing	teEther
6. Arithmetic Issues	6.1	6.1.1		✓		✓			✓		✓				✓					
		6.1.2				✓			✓						✓					
	6.2	6.2.1													✓					
		6.2.2																		
	6.3	6.3.1																		
		6.3.2																		
7. Improper Access Control	7.1	7.1.1	✓			✓					✓			✓						
		7.1.2			✓						✓									
		7.1.3				✓					✓									
	7.2	7.2.1									✓									
		7.2.2									✓									
	7.3	7.3.1									✓									✓
		7.3.2									✓									
		7.3.3									✓									
		7.3.4									✓									

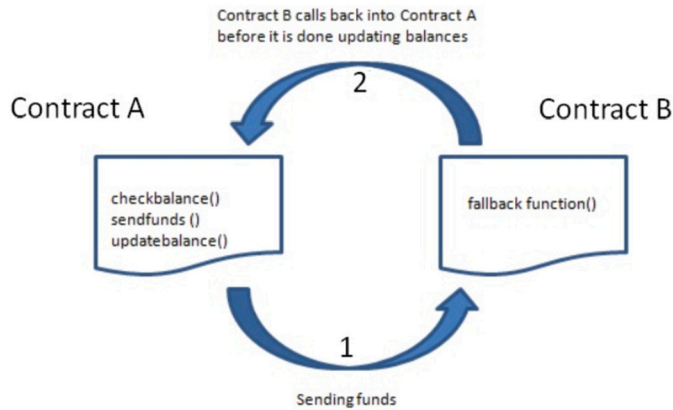


Fig. 4. The flows of interactions between two contracts.

From a historical perspective, the reentrancy attack stands out as one of the most devastating attacks in the realm of Solidity smart contracts. This particular attack has resulted in substantial financial losses, amounting to hundreds of millions of dollars over the past few years. We can cite:

- *The DAO hack*: a reentrancy attack on the DAO in 2016 stole 3.6 million Ether (\$60 million) and forced Ethereum to fork into two chains: Ethereum Classic and Ethereum [42].
- *dForce*: a hacker stole \$25 million in bitcoin and ether from dForce's lending protocol, leaving it with less than 1% of its assets [43]. In fact, a hacker used the combination of ERC777 tokens and DeFi smart contracts to perform a reentrancy attack. Thanks to the call-back mechanism, he was able to issue and withdraw ERC777 tokens repeatedly, taking advantage of the delay in balance updates [44].
- *Grim Finance*: a hacker stole approximately \$30 million in tokens from the DeFi Grim Finance project by exploiting a reentrancy flaw in the *safeTransferFrom* function of the smart contract. This function was not updating its internal state before transferring tokens to an external contract, which could call the function back and cause errors [45].

It's worth noting that the reentrancy vulnerability is divided into two subcategories:

- **unsafe credit transfer (1.1.1)**: reentrancy is a critical vulnerability in smart contracts, brought to light after the DAO attack in 2016. It allows attackers to manipulate balances by executing credit transfers before the completion of a previous call. In short, this vulnerability

enables unintentional fund transfers, facilitating malicious attacks. The various designations, "Reentrancy" [46], "Reentrancy with balance change" [47], "SWC-107 reentrancy" [48].

- **unsafe system state changes (1.1.2)**: this vulnerability is similar in nature to 1.1.1, with the main difference being that no credit is involved and therefore, there is no impact on users' funds. Due to the way the contract is coded, a call that reaches the vulnerable contract before a previous one has ended may allow an attacker to place the program in an unexpected state, leading to various effects depending on the type of contract involved, including performance or availability issues. This vulnerability is also known in the literature as "ReentrancyNoETH" [49], "Reentrancy" [50], "Reentrancy without balance change" [47], or "SWC-107 reentrancy" [48]. These two subcategories illustrate different ways in which reentrancy can be exploited for malicious attacks in smart contracts.

4.3.2. Delegatecall to untrusted callee (1.6)

The use of the `delegateCall` function to invoke untrusted contracts is a dangerous practice. It exposes the calling contract to multiple vulnerabilities. The called contract can potentially modify sensitive variables of the calling contract, such as `msg.data` (the function arguments) and `msg.sender` (the address that initiated the call).

- **A striking example: the Parity hack**

The Parity hack in 2018 perfectly illustrates the dangers of this practice. By exploiting a flaw in the `delegateCall` function of the Parity multi-signature wallet contract, attackers were able to take control of users' wallets. They achieved this by calling a malicious contract via `delegateCall`, which allowed them to override the intended functionality and reset the ownership and usage parameters of users' wallets, leading to significant financial losses.

- **A range of terminologies for a single danger**

This vulnerability is known by various names, highlighting its different aspects:

- Unrestricted delegate call [49]: emphasizes the lack of checks when selecting the target address.
- Dangerous delegate call [19,51]: highlights the potential harm of `delegateCall` calls.
- Unchecked delegate call function [52]: emphasizes the need for thorough validation before executing `delegateCall` calls.
- Code injection [31]: compares the attack to injecting malicious code into the calling contract.
- Control-flow hijack [15]: describes the attacker's ability to hijack the contract's execution flow.
- Delegated call [53–55]: the generic term for the functionality.

- Cross-program invocation [56]: highlights the interaction between two separate contracts.
- Tainted delegatecall [57]: indicates the possibility of malicious data injection via the delegateCall call.
- SWC-112, Delegatecall to untrusted callee [48]: a standardized classification in the Smart Contract Weakness Classification system.

4.3.3. Incorrect use of event blockchain variables for time (5.1.1)

Smart contracts are susceptible to a critical security flaw when they rely on specific details within a block (like timestamp, miner address, block number, mining difficulty, or gas limit) to control the sequence of events. This vulnerability allows miners, who have some control over block creation, to potentially manipulate the contract's behavior.

• Multiple Names, One Problem

This issue has been recognized and documented under various names in research papers:

- Block state dependence [46], [15]: this term directly highlights the contract's dependence on the state of the block for its logic.
- Time restrictions [58]: this name emphasizes the vulnerability's connection to using timestamps for controlling events.
- Blockchain effects time dependency [59]: this term focuses on how the timing of events on the blockchain is affected by this dependency.
- Timestamp dependence [60,55] and many others: this is a specific example where the contract relies solely on timestamps for control.

• Additional terminology

Several other terms describe this vulnerability with a specific focus:

- Block number dependence [51,61]: this term highlights the reliance on the block's position in the chain.
- Race condition [18]: this term describes a situation where the outcome depends on the order in which transactions are processed, which miners can influence.

• Other relevant concepts

- BlockTimestamp [62]: this is another way of referring to timestamp dependence.
- Time dependency [62]: this is a broader term encompassing both timestamp and potentially other time-related dependencies.
- Event-ordering (EO) bugs [63]: this term emphasizes the issue's impact on the intended order of events within the contract.
- SWC-116: block values as a proxy for time [48]: this refers to a specific coding weakness documented in security standards for smart contracts.

By understanding these various names and related concepts, developers and auditors can better identify and address this critical vulnerability in smart contract design.

4.3.4. Integer underflow (6.1.1)

The integer underflow attack is a common vulnerability in Ethereum smart contracts and other computer systems that manipulate integers. This vulnerability occurs when the result of a subtraction leads to a number less than zero, and that number is then used in calculations, operations, or conditions unexpectedly.

In the context of Ethereum smart contracts, this vulnerability can have serious consequences. For example, if a smart contract uses a variable to track an account balance and that variable is subtracted without adequate verification, an integer underflow could result in a negative balance, potentially allowing an attacker to withdraw funds from the contract unauthorized.

This vulnerability was one of the causes of the notorious attacks against Ethereum smart contracts between 2016 and 2018, highlighting the importance of addressing these security issues in blockchain [5].

• Other names

This vulnerability is also known by various names in the literature, including:

- Integer Bug [15]
- Integer Underflow [64]
- Integer Overflow/Underflow [65]
- Arithmetic Bugs [66]
- Underflow [62]
- Integer Overflow and Underflow [52]
- Arithmetic Issues [53]
- Overflow/Underflow [60]
- SWC-101: Integer Overflow and Underflow [48]

This reformulation avoids repetitive terms and focuses on a clear explanation for a better understanding of the issue.

4.4. Less studied or unstudied vulnerabilities

The analysis of Fig. 3 reveals a concerning distribution of OpenSCV vulnerabilities:

- **Never studied (0 studies):** the investigation of these unstudied vulnerabilities should be prioritized to understand their potential impact. It is essential to design solutions for these vulnerabilities and disseminate information to protect the OpenSCV community from possible risks. A focused effort is required to bring attention to these vulnerabilities and start addressing them in future research.
- **Studied once (1 study):** vulnerabilities with only one study need further investigation to evaluate their real-world relevance. Future research should aim to deepen the analysis of these vulnerabilities, assess their potential risks more thoroughly, and develop practical solutions to mitigate them.
- **Studied twice (2 studies):** while slightly more studied, these vulnerabilities still require further research to gain a comprehensive understanding of the threats they pose. A more in-depth analysis will help quantify the risk they represent and identify countermeasures to prevent exploitation.
- **Studied three times or more (3 studies or more):** vulnerabilities with multiple studies indicate ongoing research efforts, but further work is necessary to refine solutions and ensure their robustness across different use cases and environments.

This distribution highlights the vulnerabilities that require urgent attention from the OpenSCV community. To address these gaps and enhance security, the following actions are crucial:

- **Prioritize research on unstudied vulnerabilities:** a deeper focus on unstudied vulnerabilities is critical. Research teams should prioritize these vulnerabilities and explore them in more detail to understand their implications better.
- **Creation of a centralized database:** compile all available information on OpenSCV vulnerabilities, including the number of studies and references to existing publications. This centralized resource will facilitate more efficient collaboration and help researchers avoid duplicating efforts.
- **Developing tools and resources:** it is essential to develop tools that assist in identifying and mitigating these vulnerabilities in real-world applications. Providing such resources to developers will improve the overall security posture of OpenSCV implementations.
- **Promote collaborative research:** encouraging collaboration among security researchers is essential to addressing these vulnerabilities. By sharing knowledge, tools, and expertise, the OpenSCV community can accelerate progress toward better security.

The creation and use of an OpenSCV vulnerability taxonomy can bring numerous benefits to the OpenSCV community. This taxonomy will improve the understanding of vulnerabilities, help prioritize research and development efforts, facilitate collaboration and information sharing,

and enhance risk communication within the community. Additionally, by focusing on the less studied or unstudied vulnerabilities, this taxonomy will ensure that important gaps are filled and critical issues are addressed promptly.

5. RQ4: What metrics are used to evaluate the quality and effectiveness of security testing for Ethereum smart contracts?

Several metrics are used to assess the effectiveness of security testing methods for Ethereum smart contracts.

- **Detection capability**
 - **Precision:** this metric reflects the percentage of identified issues that are actual vulnerabilities. High precision ensures the focus is on genuine security risks [11].
 - **Recall:** this metric represents the proportion of actual vulnerabilities that the testing method successfully detects. High recall indicates a comprehensive ability to find security flaws [11].
 - **True positives (TP):** the number of correctly identified vulnerabilities. High TP indicates successful detection of real issues [13].
 - **False positives (FP):** the number of identified issues that are not actual vulnerabilities. Low FP reduces wasted time investigating non-existent threats [13].
- **Efficiency and performance**
 - **Time:** measured in seconds or minutes, it reflects how long it takes to test a contract. Faster testing allows for quicker security assessments [14].
 - **Test cases generated:** the number of test cases created during fuzzing. Higher numbers increase the likelihood of exploring diverse scenarios and uncovering vulnerabilities [13].
 - **Coverage:** this metric can refer to:
 - * **Branch coverage:** the percentage of code branches executed during testing. Higher coverage suggests a more thorough exploration of the code [13].
 - * **Instruction coverage:** the percentage of instructions executed during testing. High coverage implies a deeper examination of the contract's logic [14].
- **Real-world impact**
 - **Number of vulnerabilities found:** the total number of vulnerabilities identified in real-world smart contracts. This demonstrates the tool's effectiveness in practical situations [11].
 - **Vulnerability types:** the specific types of vulnerabilities detected (e.g., reentrancy, delegation flaws). A wider range of vulnerability detection indicates a more robust testing approach [11].
- **Additional considerations**
 - **Memory usage:** excessive memory consumption can hinder the testing process. Tools with efficient memory management are preferred [14].
 - **Ease of use:** a user-friendly interface and clear documentation simplify the testing process for developers.

5.1. Evaluation of fuzzing testing tools

In order to evaluate the performance of smart contract fuzzing tools, we conducted a comprehensive analysis of several popular tools, including xFuzz, IR-Fuzz, ItyFuzz, SMARTIAN, DEPOSafe, Ethrolic, GasFuzzer, Magic Mirror, RLF, Harvey, and ConFuzzius. We extracted relevant parameters and evaluation results from research papers and official documentation to compare their effectiveness in terms of code coverage, vulnerability detection, and overall performance.

- **FunFuzz** is an advanced tool for detecting vulnerabilities in smart contracts, surpassing existing fuzzers in both efficiency and effectiveness. It targets specific vulnerabilities such as reentrancy, block

dependency, and gasless send, providing a more focused approach to fuzz testing. By reducing the scope of exploration to high-risk functions, FunFuzz efficiently identifies critical vulnerabilities that might be missed by traditional fuzzers. Extensive testing on 170 real-world smart contracts has demonstrated its ability to uncover serious security flaws, making it an invaluable tool for ensuring the safety of DApps.

- **xFuzz** demonstrates exceptional precision and recall, particularly for reentrancy, delegation, and tx-origin vulnerabilities. It significantly outperforms sFuzz in terms of time savings and vulnerability detection.
- **IR-Fuzz** exhibits high precision for all vulnerability types, identifying over 485 vulnerable contracts with minimal false positives. It achieves impressive branch coverage and outperforms other methods in terms of efficiency.
- **ItyFuzz** excels in instruction coverage, surpassing SMARTIAN on benchmark datasets. It effectively identifies real-world vulnerabilities and demonstrates superior response times for on-chain audits.
- **SMARTIAN** enhances fuzzing effectiveness through static and dynamic analyses, outperforming other tools in bug detection and instruction coverage. It scales well to large-scale contracts and identifies various vulnerability types.
- **DEPOSafe** achieves 100% precision in identifying vulnerabilities related to false deposits in ERC-20 smart contracts. Its high recall indicates its ability to uncover most real positives.
- **Ethrolic** proves effective in identifying a wide range of vulnerabilities, including Integer Overflow (IOF), Integer Underflow (IUF), Bad Randomness (BR), Re-entrancy (RE), Locked Ether (LE), Unhandled Exceptions (UE), Denial of Service (DoS), Short Address Attack (SA), Race Condition (RC), and Shadow Memory (SM). It detects novel vulnerabilities and demonstrates comprehensive protection capabilities.
- **GasFuzzer** surpasses ContractFuzzer in detecting Exceptions Disorder vulnerabilities by 28%. Its gas-greedy strategy and gas-leveling technique enhance vulnerability detection, particularly for gas-related issues.
- **Magic Mirror** utilizes constraint solving, state exploration, and combinatorial testing to achieve high code coverage and vulnerability detection rates.
- **RLF** outperforms Mythril and SmarTest in terms of vulnerability detection efficiency, identifying 98% of vulnerabilities within 30 minutes. It effectively detects vulnerabilities in real-world contracts.
- **Harvey** significantly reduces the time required to detect bugs and improves instruction coverage by utilizing input prediction and demand-oriented sequence fuzzing.
- **ConFuzzius** achieves superior code coverage compared to ILF and SFUZZ, particularly for large contracts. It effectively detects various vulnerability types, including Assertion Failure, Integer Overflows, Reentrancy, and Unsafe Delegatecall.

5.2. Evaluation of mutation testing tools

Mutation testing is a technique for assessing the effectiveness of test suites by evaluating their ability to detect modified versions of the original code, known as mutants. SuMo, a mutation testing tool for smart contracts, was evaluated using two dApp projects: Ether Crowdfunding and bionic-event-dapp. The results demonstrated SuMo's effectiveness in generating and detecting mutants for both projects. Key metrics included:

- **Total mutants:** the total number of mutants generated for each project.
- **Stillborn mutants:** mutants that were either syntactically incorrect or did not affect the program's behavior.
- **Equivalent mutants:** mutants that were not detected by the test suite, even though they should have been.

- Mutation score: the percentage of mutants that were successfully killed by the test suite.

The results showed that SuMo generated a substantial number of mutants for both projects and effectively detected a majority of them. The mutation scores of 47.73% for Ether Crowdfunding and 58.7% for bionic-event-dapp indicate the tool's ability to identify potential vulnerabilities.

5.3. Evaluation of symbolic execution tools

- SMARTTEST:

The evaluation of SMARTTEST is based on several metrics to compare its effectiveness with other similar tools in detecting vulnerabilities in smart contracts. Here are the key results:

- Number of vulnerable transaction sequences found: SMARTTEST identified a significant number of vulnerable transaction sequences, demonstrating its effectiveness in detecting flaws.
- Number of validated vulnerable transaction sequences: a high percentage of the vulnerable sequences identified by SMARTTEST were validated as truly vulnerable, indicating the tool's precision.
- Execution time: the time required for SMARTTEST to analyze a set of contracts was 6 hours and 7 minutes for the CVE data, indicating in-depth analysis, but requiring a significant time investment.
- Number of identified CVE vulnerabilities: SMARTTEST was able to identify a significant number of CVE vulnerabilities, confirming its ability to detect known flaws.
- Vulnerability detection success rate: SMARTTEST's success rate in identifying real vulnerabilities was high, demonstrating its reliability.
- Number of false positives: SMARTTEST presented a low number of false positives, minimizing erroneous alerts.
- Number of false negatives: the number of false negatives was also low, ensuring comprehensive detection of vulnerabilities.
- Analysis cost in terms of computing resources: although SMARTTEST requires significant resources in terms of CPU time and memory, its performance compensates for this investment.
- Number of successfully analyzed contracts: SMARTTEST successfully analyzed a considerable number of contracts without errors or failures, highlighting its robustness.
- Zero-day vulnerability discovery rate: SMARTTEST demonstrated its ability to identify unknown vulnerabilities, reinforcing its usefulness for smart contract security.

- RA:

The RA tool was evaluated based on various criteria, showing notable performance in detecting reentry-type vulnerabilities. Here is a summary of the results:

- Ability to Perform Inter-Contract Control Flow Analysis: RA surpassed other tools in analyzing reentry attacks based on creation and cross-function calls, demonstrating its expertise.
- Precision in Vulnerability Detection: RA achieved a true positive rate of 100% and a false positive rate of 0%, indicating precise detection of vulnerabilities without false alarms.
- Computational performance: despite an increase in analysis time with the number of function calls, RA maintained reasonable deadlines even for complex contracts.
- Comparison with other tools: RA outperformed some static analysis tools in detecting reentry attacks, offering performance comparable to dynamic analysis tools.
- Computational complexity: the analysis of RA's mathematical complexity suggests efficient analysis for user-developed contracts, although improvements are possible.

- SAILFISH:

The evaluation results of SAILFISH highlight its performance in detecting vulnerabilities, especially reentry attacks and TOD. Here is an overview of the obtained data:

- Vulnerability detection: SAILFISH reported a high percentage of secure contracts, with a limited number of warnings and expired contracts, demonstrating its effectiveness.
- Comparison with other tools: SAILFISH presented lower vulnerability rates compared to some tools for reentry and TOD, emphasizing its reliability in detecting flaws.
- Performance analysis: the average analysis times for different data sets show the relative efficiency of SAILFISH, although it can be optimized for faster analyses.
- Exploitability of bugs: SAILFISH discovered a significant number of zero-day bugs, some of which were exploitable, highlighting its importance for smart contract security.
- Comparison with SEREUM: SAILFISH identified vulnerabilities not reported by other tools, demonstrating its ability to detect unique flaws.

- WANA:

The evaluation of WANA revealed positive results regarding its ability to detect various vulnerabilities in Ethereum smart contracts, with notable efficiency and speed. Here are the key points:

- Vulnerability detection: WANA identified all detected vulnerabilities in the studied smart contracts, without any false negatives, indicating its precision.
- WANA's efficiency: the average analysis time per smart contract was very short, at only 0.08 seconds, highlighting the tool's speed and efficiency.
- Other metrics: although specific metrics were not mentioned, WANA's overall performance is positive, with an ability to identify known and unknown vulnerabilities. These evaluations demonstrate the diversity of tools available for smart contract security, each having its strengths and potential areas for improvement, but all contributing to enhancing the reliability and robustness of blockchain applications.

- teEther:

- TeEther was utilized to analyze a snapshot of the Ethereum blockchain taken on November 30, 2017. This analysis identified a total of 784,344 contracts. After eliminating duplicates, the dataset was refined to include 38,757 unique contracts, which were used for further investigation.
- TeEther was executed on 38,757 contracts, with a 30-minute time limit for each contract's CFG reconstruction and exploit generation.
- The system used for the experiments featured an Intel Xeon E5-2660 processor with 16 threads and 192 GB of memory.
- 85.65% (33,195 contracts) of the analysis was completed within the time limit.
- TeEther successfully generated 815 exploits (2.10% of analyzed contracts).
- Critical paths were detected in 8,049 contracts, but no exploits were possible in most of them.
- Exploits targeted vulnerabilities in CALL and SELFDESTRUCT functions.
- 855 exploits targeted 815 contracts, with 664 independent exploits affecting 630 unique contracts and 1,731 contract accounts.
- Validation: 88.41% (1564 exploits) successfully extracted funds on a private test network, while 5.26% (93 exploits) failed, and 6.33% (112 exploits) failed during the update stage due to mismatched storage assumptions.
- Manual analysis of 44 contracts identified four categories of vulnerabilities: erroneous visibility, constructor errors, semantic confusion, and logic flaws.

- Most vulnerabilities (except logic flaws) were linked to Solidity’s design, with recommendations to make functions internal by default and introduce a specific keyword for constructors.
- DefectChecker:
 - **Above average performance:** DefectChecker successfully detected defects in smart contracts, outperforming other popular tools like Oyente, Mythril, and Securify in detection scores. It achieved particularly high results when tested on an existing dataset.
 - **Scope of the analysis:** the tool was executed on 165,621 distinct smart contracts from the Ethereum blockchain and revealed that 15.89% of contracts contained at least one of the eight types of defects identified by DefectChecker.
 - **Defect identification:** DefectChecker used symbolic execution to analyze the bytecodes of smart contracts. The defects detected were primarily security issues related to the bytecode and were attributed to design errors in the smart contract code.
 - **False positives and false negatives:** the tool showed limitations in detecting certain defects with false positives and false negatives, particularly for defects such as null condition (NC) and data unused error indicator (DuEI).
 - **Room for improvement:** a suggested improvement is to integrate an SMT solver to refine the accuracy of the analysis. Although this may increase execution time, adding an SMT solver would better handle errors, particularly concerning conditional expressions in loops and the detection of unreachable paths in the bytecode.

6. RQ5: What are the current challenges and limitations in the field of security testing for Ethereum smart contracts?

Introduction to challenges and limitations: smart contracts on the Ethereum blockchain are seeing increased adoption across various sectors, but their security remains a major concern. Code errors can lead to financial losses and security breaches, posing challenges such as the growing complexity of contracts, diverse usage scenarios, and the need for robust security standards. Understanding and addressing these challenges is crucial to ensuring the reliability of Ethereum smart contracts.

- **Specific challenges in security testing for Ethereum**
 - **Immutability:** one of the major challenges in the development and security testing of smart contracts lies in the immutability of the blockchain. This characteristic implies that everything recorded on the blockchain is permanent and cannot be altered. Consequently, any vulnerability or programming error that goes undetected before contract deployment becomes difficult, if not impossible, to rectify without compromising the system’s integrity and trust. Unlike traditional software development, where iterations and corrections are possible throughout the process, smart contracts have limitations in terms of modifications post-deployment. This means developers must exercise extreme vigilance during security testing to identify and resolve all vulnerabilities early in the development stages, as these vulnerabilities can transform into potential exploits once the contract is active on the blockchain. Thus, security testing must be comprehensive and thorough prior to deployment to minimize the risks of vulnerabilities and future exploits, while also considering the limitations of iterative development to make necessary corrections early in the process [67].
 - **The complexity and obscurity of the Solidity language semantics [68]:**

The complexity and obscurity of Solidity’s semantics contribute to several challenges and potential vulnerabilities in the development and security of smart contracts. Here’s how this manifests:

- * **Partial understanding:** solidity is a language specific to the Ethereum blockchain, designed for writing smart contracts. Its syntax and concepts can be challenging to grasp for developers not familiar with decentralized programming. As a result, even experienced programmers may have a partial understanding of how certain features or structures in Solidity actually work.
- * **Risk of design errors:** the obscurity of the semantics can lead to design errors. For example, a developer might misinterpret how a function or variable operates, leading to logical errors in the smart contract.
- * **Security vulnerabilities:** ambiguities or subtleties in Solidity’s semantics can open doors to security vulnerabilities. For instance, a misunderstanding of permission management or fund transfer functions could result in security flaws such as reentrancy attacks, capacity overflows, or trap contracts.
- * **Contract complexity:** complex smart contracts written in Solidity can be challenging to audit and test thoroughly due to the complexity of their semantics. This can result in gaps in security testing and leave potential vulnerabilities undiscovered.
- * **Language evolution:** as Solidity is an evolving language, new features and changes in semantics may occur. This necessitates ongoing updates to knowledge and development practices, which can be a challenge for developers. In summary, Solidity’s obscure semantics make understanding and developing smart contracts more complex, increasing the risk of design errors and security vulnerabilities. Developers must therefore exercise great caution and use best development practices to mitigate these risks as much as possible.
- **Scalability [69]: How to effectively test thousands of deployed smart contracts.** The scalability challenge of Ethereum refers to the network’s ability to process a large number of transactions efficiently and quickly while maintaining reasonable transaction costs. Ethereum, as a public blockchain, faces scalability challenges due to its original design, which prioritizes decentralization and security over speed and scalability.

Here are some of the main scalability challenges that Ethereum faces:

 - * **Limited throughput:** Ethereum is limited in terms of transactions per second (TPS). Its current capacity is approximately 15 to 30 TPS, which is insufficient to support large-scale applications.
 - * **Variable transaction costs:** during network congestion, transaction fees can significantly increase, making microtransactions or frequent transactions impractical.
 - * **Confirmation time:** the time required for a transaction to be confirmed on the Ethereum blockchain can vary depending on network load, leading to delays in transactions.
 - * **Resource management:** Ethereum nodes must manage a significant amount of data to stay synchronized with the blockchain, which can be challenging for low-performing nodes or individual users.
 - * **Complex smart contracts:** complex smart contracts and resource-intensive operations can increase network load and reduce its ability to process new transactions quickly.
- **Complexity of interactions [70]: contracts interact with one another, complicating tests**

- * **Multiple interactions:** smart contracts can interact with each other in complex ways, calling and being called by other contracts in scenarios where multiple contracts are involved simultaneously. Attack Risks: Interactions between smart contracts open the door to attack risks, where malicious contracts can be designed to exploit vulnerabilities in other contracts or induce unexpected behaviors.
 - * **Prediction difficulty:** it is often challenging to predict how multiple contracts will interact with each other, especially when these interactions can be triggered by external events or variable data.
 - * **Global understanding:** understanding the full scope of possible interactions between multiple smart contracts is a challenge in itself, as it requires a deep analysis of dependencies, data flows, and triggering conditions for interactions.
 - * **Security implications:** interactions between smart contracts can lead to security issues such as data leaks, overflow attacks, unexpected reentrancies, etc.
- **Limitations in Ethereum security testing [71]**
 - **Incomplete test coverage:**
Even thorough testing cannot guarantee 100% code coverage. Gaps may exist in untested areas or unexpected input scenarios. Reliance on unit tests and fuzzing may limit the detection of more complex issues related to the interaction between different contracts or race conditions.
 - **Difficulty in identifying logical flaws:**
Certain logical flaws, such as reasoning errors or inconsistencies in business rules, may be hard to detect by automated testing. Formal verification can aid in identifying these flaws, but requires expertise and a precise formal specification of the contract.
 - **Evolving threat landscape:**
New vulnerabilities and attack vectors emerge constantly, making it challenging to update tests to keep up with the evolving threat. Tests must be continuously updated and adapted to ensure they remain effective against the latest threats.
 - **Cost and time:**
Thorough security testing, especially formal verification, can be expensive and time-consuming. A balance must be found between testing rigor and the resources available for contract development and maintenance.
 - **Human factors:**
Human errors in writing tests or interpreting results can compromise the effectiveness of security testing. Close communication and collaboration between developers, testers, and security auditors are crucial. In summary, smart contract security testing is essential to protect dApps from vulnerabilities and attacks, but it has limitations that must be considered. It is important to adopt a multi-layered approach that combines different testing techniques and ensures tests are continually updated and adapted to the evolving threat landscape.

7. Discussion

Our literature review has identified several security testing techniques used to evaluate Ethereum smart contracts. These techniques include fuzzing, symbolic execution, mutation testing, and MT. Fuzzing, which represents a significant portion of research and development efforts, involves providing invalid, unexpected, or random inputs to a program to detect bugs and vulnerabilities. Symbolic execution explores all possible execution paths of a program using symbolic values, thereby detecting bugs that are not detectable by concrete execution. Mutation testing deliberately injects modifications into a program's source code

to evaluate whether existing tests can detect these mutations and ensure code robustness. Finally, MT relies on MRs to validate system behavior after transformations of input data.

Each of these techniques has specific advantages and limitations. Fuzzing offers efficient automation and code coverage, but can generate false positives and be time-consuming. Symbolic execution allows for in-depth analysis of program execution paths, but can be resource-intensive. Mutation testing improves test coverage and code quality but can be challenging to interpret. Finally, MT reduces reliance on test oracles and detects defects not detectable by other techniques, but requires the identification of meaningful MRs.

The analysis methodology used to evaluate vulnerabilities in the Ethereum smart contracts, along with the introduction of the OpenSCV taxonomy, has provided valuable insights into the most common vulnerabilities and areas requiring further research. The most studied vulnerabilities include reentrancy attacks, delegateCall to unreliable callers, and incorrect use of the blockchain event variables for time, highlighting their significant impact on the smart contract security. These vulnerabilities have historical examples, such as The DAO hack and the Parity hack, highlighting the real consequences of these security flaws.

Additionally, the analysis has identified less studied or unstudied vulnerabilities, underscoring the need for targeted research and collaboration efforts within the OpenSCV community. Recommendations to address these gaps include creating a centralized database, developing tools for vulnerability identification and correction, and promoting collaborative research among security experts. The adoption of the OpenSCV taxonomy promises better risk communication, improved prioritization of security efforts, and enhanced collaboration to strengthen smart contract security in the blockchain ecosystem.

Several criteria are used to evaluate the quality and effectiveness of security tests for Ethereum smart contracts. These criteria include detection capability, efficiency and performance, real-world impact, and other considerations such as memory usage and ease of use. To evaluate fuzzing tools, criteria such as code coverage, vulnerability detection, and overall performance were analyzed for tools like xFuzz, IR-Fuzz, ItyFuzz, SMARTIAN, DEPOSafe, Ethrolic, GasFuzzer, Magic Mirror, RLF, Harvey, and ConFuzzius. These tools exhibit varied performance in terms of accuracy, code coverage, and vulnerability detection, with specific strengths and weaknesses for each tool. For the evaluation of mutation testing tools, SuMo was used to assess its ability to generate and detect mutants in specific dApp projects. The results showed that SuMo was effective in detecting a majority of mutants, with mutation scores indicating its ability to identify potential vulnerabilities. Regarding symbolic execution tools like SMARTEST, RA, SAILFISH, and WANA, criteria such as vulnerability detection, execution time, accuracy, and ability to identify unknown vulnerabilities were evaluated. These tools demonstrated positive performance in detecting various vulnerabilities, although some may require improvements to optimize efficiency and analysis speed. In summary, the evaluation of these tools highlights their diversity and varied performance in detecting vulnerabilities in Ethereum smart contracts, emphasizing the importance of choosing tools tailored to the specific security needs of developers and smart contract auditors.

The article highlights several major challenges and limitations in the field of security testing for Ethereum smart contracts on the Ethereum blockchain. One of the main challenges lies in the immutability of the blockchain, which makes rectification after deploying a smart contract difficult, if not impossible, without compromising system integrity. This characteristic demands extreme vigilance during testing to identify and correct all vulnerabilities early in the development stages, as an undetected flaw can turn into an exploitable loophole once active on the blockchain.

Another significant challenge lies in the complexity and opacity of the Solidity language syntax used to write smart contracts. This complexity makes understanding Solidity's features and structures difficult for developers, leading to design errors and security vulnerabilities. Furthermore, Solidity's constant evolution requires frequent updates

to development practices to adapt to new changes and reduce risks. Ethereum's scalability also poses a major challenge, with limitations in terms of TPS, variable transaction fees, and variable confirmation times. This limitation can hinder the deployment of large-scale applications on the Ethereum blockchain, necessitating innovative solutions to improve scalability while maintaining security and decentralization.

The complexity of interactions between smart contracts represents another obstacle, with potential risks of malicious attacks and unexpected behaviors. The difficulty in predicting these interactions and understanding their overall impact makes security testing more complex, requiring in-depth analysis of dependencies and data flows between contracts.

Finally, limitations of security testing itself, such as incomplete code coverage, difficulty in identifying logical flaws, and the costs associated with rigorous testing, underscore the need for a multi-level approach to ensure Ethereum smart contract security. This approach should combine different testing techniques, be adaptable to evolving threats, and encourage close collaboration between developers, testers, and security auditors.

8. Future works

Future work in this field could focus on several aspects to enhance the security of Ethereum smart contracts:

- **Diversified case studies:** enriching case studies by including a wider range of smart contracts across various sectors such as finance, healthcare, education, etc. This will help understand sector-specific vulnerabilities and adapt best security practices accordingly.
- **Exploring new blockchains:** besides Ethereum, exploring other blockchains like Binance Smart Chain, Polkadot, and Cardano to understand differences in security protocols and identify the best approaches to securing smart contracts on these alternative platforms.
- **Improving testing techniques:** developing more advanced testing techniques, such as integrating artificial intelligence and machine learning for automated vulnerability detection, enhancing code coverage, and reducing false positives in security test results.
- **Securing sensitive data:** focusing on safeguarding sensitive data stored in smart contracts using advanced encryption techniques and strengthening key management protocols to mitigate risks of privacy breaches.
- **Development of specialized security tools:** designing and developing specialized security tools for smart contract developers, such as automated security scanners, vulnerability simulation platforms, and sandbox test environments for more secure contract evaluation.
- **Enhancing collaboration and awareness:** encouraging collaboration among researchers, developers, and security auditors to share knowledge, best practices, and lessons learned. Raising further awareness among developers about secure programming best practices for smart contracts.
- **Adoption of security standards:** promoting the adoption of specific security standards for Ethereum smart contracts by developing guidelines and recommendations to ensure optimal security in contract development and deployment. By combining these various research and development aspects, it is possible to significantly enhance the security of Ethereum smart contracts and strengthen user trust in the blockchain ecosystem.

9. Conclusions

Research on Ethereum smart contract security highlights a diverse range of techniques and methodologies aimed at ensuring the reliability and security of smart contracts. Among the main techniques discussed are fuzzing, symbolic execution, mutation testing, and MT.

Fuzzing, as an automated testing technique, is highly effective in identifying bugs and vulnerabilities by providing unexpected inputs to smart contracts. It offers automation, code coverage exploration, and

bug detection capabilities, but may suffer from false positives and execution time issues.

Symbolic execution, a static analysis technique, explores all possible execution paths of a program, thereby detecting bugs that traditional methods may miss. While powerful, it can be resource-intensive and challenging to implement for complex contracts.

Mutation testing involves injecting mutations into the source code to evaluate the robustness of existing tests. It improves test coverage and code quality, but can be costly and challenging to interpret results.

MT, based on defining MRs, offers an effective approach to testing smart contracts by reducing reliance on test oracles and generating more effective test cases. However, it depends on identifying meaningful relations and may be difficult to automate fully.

By combining these techniques and tools, developers can enhance the overall quality and reliability of their smart contracts, thereby reducing the risk of vulnerabilities and ensuring a more secure blockchain ecosystem.

The analysis of security testing tools and frameworks for smart contracts reveals that fuzzing is a predominant technique, with several specialized tools available for grey-box testing, reflecting the industry's emphasis on automated testing with random input data.

On the other hand, mutation testing tools are less prevalent, indicating that this approach might not be as widely adopted or prioritized in the context of smart contract security testing.

Symbolic execution tools are represented, but to a lesser extent compared to fuzzing, suggesting a more selective use of this technique possibly due to its resource-intensive nature and complexity in analyzing all possible execution paths.

MT, while essential for verifying transformations of input data, is relatively less common, with only one tool listed for grey-box testing. This could imply a need for further development or awareness of this approach within the smart contract security testing community.

In conclusion, the availability and usage of different testing techniques vary, with fuzzing being the most prominent, followed by symbolic execution, mutation testing, and MT. The choice of tools and techniques should align with the specific requirements and complexities of the smart contract being tested, highlighting the importance of informed decision-making in security testing practices.

The detailed analysis methodology used to assess smart contract vulnerabilities, along with the presentation of the OpenSCV taxonomy and its benefits, provides a comprehensive framework for understanding and addressing security challenges in smart contracts.

The results of the vulnerability analysis offer a visual representation of the research efforts dedicated to each vulnerability category between 2020 and 2023, helping to identify critical areas in smart contract security and shedding light on under-researched vulnerabilities that deserve increased attention.

By leveraging the OpenSCV taxonomy and prioritizing research efforts based on identified vulnerabilities, the smart contract community can enhance security practices, mitigate risks, and foster a more secure and trustworthy smart contract ecosystem.

The evaluation of various security testing tools for Ethereum smart contracts provides valuable insights into their effectiveness in identifying vulnerabilities and improving overall security. These evaluations showcase the diversity of tools available for smart contract security, each with its strengths and capabilities.

Combining these tools and leveraging their unique features can significantly enhance the reliability, robustness, and trustworthiness of Ethereum smart contracts, ultimately contributing to a more secure blockchain ecosystem.

These overall conclusions provide a comprehensive overview of the challenges and opportunities in the field of Ethereum smart contract security, emphasizing the importance of an integrated and balanced approach to testing and security techniques to ensure trust and security in the smart contract ecosystem.

CRediT authorship contribution statement

Marwa Mnasri: Writing – original draft, Visualization, Validation, Supervision, Resources, Methodology, Data curation, Conceptualization. **Afef Jmal Maâlej:** Writing – review & editing, Validation, Supervision, Methodology, Investigation, Conceptualization. **Mohamed Jmaiel:** Visualization, Supervision.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] LabCFTC, A primer on smart contracts, https://www.cftc.gov/sites/default/files/2018-11/LabCFTC_PrimerSmartContracts112718.pdf, 2018. (Accessed 15 August 2023).
- [2] H. Zhou, A. Fard, A. Makanju, The state of Ethereum smart contracts security: vulnerabilities, countermeasures, and tool support, *J. Cybersecur. Priv.* 2 (2) (2022) 358–378, <https://doi.org/10.3390/jcp2020019>.
- [3] N. Quiroz, Ethereum development documentation, <https://ethereum.org/en/developers/docs/>, 2023. (Accessed 15 August 2023).
- [4] B. Hu, Z. Zhang, J. Liu, et al., A comprehensive survey on smart contract construction and execution: paradigms, tools, and systems, *Patterns* 2 (2) (2021) 100179, <https://doi.org/10.1016/j.patter.2020.100179>.
- [5] P. Praitheeshan, L. Pan, J. Yu, et al., Security analysis methods on Ethereum smart contract vulnerabilities: a survey, *arXiv preprint, arXiv:1908.08605*, 2020.
- [6] J. Xu, F. Dang, X. Ding, et al., A survey on vulnerability detection tools of smart contract bytecode, in: *Proceedings of the 2020 IEEE 3rd International Conference on Information Systems and Computer Aided Education (ICISCAE)*, IEEE, 2020, pp. 1–5, <https://doi.org/10.1109/ICISCAE51034.2020.9236931>.
- [7] R. Taş, Smart contract security vulnerabilities, *Erzincan Univ. J. Sci. Technol.* 16 (1) (2023) 196–211, <https://doi.org/10.18185/erzifed.1105551>.
- [8] P. Qian, Z. Liu, Q. He, et al., Smart contract vulnerability detection technique: a survey, *arXiv preprint, arXiv:2209.05872*, 2022.
- [9] B. Kitchenham, S.M. Charters, Guidelines for Performing Systematic Literature Reviews in Software Engineering, https://legacyfiles.elsevier.com/promis_misc/525444systematicreviewsguide.pdf, 2007. (Accessed 15 August 2023).
- [10] M. Ye, Y. Nan, H.-N. Dai, et al., FunFuzz: a function-oriented fuzzer for smart contract vulnerability detection with high effectiveness and efficiency, *ACM Trans. Softw. Eng. Methodol.* 33 (7) (2024) 1–20, <https://doi.org/10.1145/3674725>.
- [11] Y. Xue, J. Ye, W. Zhang, et al., xFuzz: machine learning guided cross-contract fuzzing, *IEEE Trans. Dependable Secur. Comput.* 21 (2) (2024) 515–529, <https://doi.org/10.1109/TDSC.2022.3182373>.
- [12] H. Feng, X. Ren, Q. Wei, et al., Magicmirror: towards high-coverage fuzzing of smart contracts, in: *Proceedings of the 2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*, IEEE, 2023, pp. 141–152, <https://doi.org/10.1109/ICST57152.2023.00022>.
- [13] Z. Liu, P. Qian, J. Yang, et al., Rethinking smart contract fuzzing: fuzzing with invocation ordering and important branch revisiting, *IEEE Trans. Inf. Forensics Secur.* 18 (2023) 1237–1251, <https://doi.org/10.1109/TIFS.2023.3237370>.
- [14] C. Shou, S. Tan, K. Sen, ItyFuzz: snapshot-based fuzzer for smart contract, in: *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA '23)*, ACM, 2023, <https://doi.org/10.1145/3597926.3598059>.
- [15] J. Choi, D. Kim, S. Kim, et al., SMARTIAN: enhancing smart contract fuzzing with static and dynamic data-flow analyses, in: *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021, pp. 227–239, <https://doi.org/10.1109/ASE51524.2021.9678888>.
- [16] R. Ji, N. He, L. Wu, et al., DEPOSafe: demystifying the fake deposit vulnerability in Ethereum smart contracts, in: *Proceedings of the 2020 25th International Conference on Engineering of Complex Computer Systems (ICECCS)*, IEEE, 2020, pp. 125–134, <https://doi.org/10.1109/ICECCS51672.2020.00022>.
- [17] G. Grieco, W. Song, A. Cygan, et al., Echidna: effective, usable, and fast fuzzing for smart contracts, in: *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ACM, 2020, pp. 557–560, <https://doi.org/10.1145/3395363.3404366>.
- [18] M. Ashouri, Etherolic: a practical security analyzer for smart contracts, in: *Proceedings of the 35th Annual ACM Symposium on Applied Computing*, Association for Computing Machinery, ACM, 2020, pp. 353–356, <https://doi.org/10.1145/3341105.3374226>.
- [19] I. Ashraf, X. Ma, B. Jiang, et al., Gasfuzzer: fuzzing Ethereum smart contract binaries to expose gas-oriented exception security vulnerabilities, *IEEE Access* 8 (2020) 99552–99564, <https://doi.org/10.1109/ACCESS.2020.2995183>.
- [20] M. Rodler, D. Paaßen, W. Li, et al., EF/CF: high performance smart contract fuzzing for exploit generation, in: *Proceedings of the 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, IEEE, 2023, pp. 449–471, <https://doi.org/10.1109/EuroSP57164.2023.00034>.
- [21] J. Su, H.-N. Dai, L. Zhao, et al., Effectively generating vulnerable transaction sequences in smart contracts with reinforcement learning-guided fuzzing, in: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE'22)*, ACM, 2022, pp. 1–12, <https://doi.org/10.1145/3551349.3560429>.
- [22] V. Wüstholtz, M. Christakis, Harvey: a greybox fuzzer for smart contracts, in: *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*, ACM, 2020, pp. 1398–1409, <https://doi.org/10.1145/3368089.3417064>.
- [23] C.F. Torres, A.K. Iannillo, A. Gervais, et al., ConFuzzius: a data dependency-aware hybrid fuzzer for smart contracts, in: *Proceedings of the 2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, IEEE, 2021, pp. 103–119, <https://doi.org/10.1109/EuroSP51992.2021.00018>.
- [24] Y. Fu, M. Ren, F. Ma, et al., EVMFuzzer: detect EVM vulnerabilities via fuzz testing, in: *Proceedings of the 2019 ACM Conference*, ACM, 2019, pp. 1110–1114, <https://doi.org/10.1145/3338906.3341175>.
- [25] X. Wang, J. Sun, C. Hu, et al., EtherFuzz: mutation fuzzing smart contracts for TOD vulnerability detection, *Wirel. Commun. Mob. Comput.* 2022 (1) (2022) 1565007, <https://doi.org/10.1155/2022/1565007>.
- [26] M. Barboni, A. Morichetta, A. Polini, SuMo: a mutation testing strategy for solidity smart contracts, in: *Proceedings of the 2021 IEEE/ACM International Conference on Automation of Software Test (AST)*, IEEE/ACM, 2021, pp. 50–59, <https://doi.org/10.1109/AST52587.2021.00014>.
- [27] S. So, S. Hong, H. Oh, Smartest: effectively hunting vulnerable transaction sequences in smart contracts through language model-guided symbolic execution, in: *Proceedings of the 30th USENIX Security Symposium*, USENIX Association, 2021, pp. 1361–1378.
- [28] D. Wang, B. Jiang, W. Chan, Wana: symbolic execution of wasm bytecode for cross-platform smart contract vulnerability detection, *arXiv preprint, arXiv:2007.15510*, 2020.
- [29] Y. Chinen, N. Yanai, J. Cruz, et al., RA: hunting for re-entrancy attacks in Ethereum smart contracts via static analysis, in: *Proceedings of the 2020 IEEE International Conference on Blockchain (Blockchain)*, IEEE, 2020, pp. 327–336, <https://doi.org/10.1109/Blockchain50366.2020.00048>.
- [30] P. Bose, D. Das, Y. Chen, et al., SAILFISH: vetting smart contract state-inconsistency bugs in seconds, in: *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2022, pp. 161–178, <https://doi.org/10.1109/SP46214.2022.9833721>.
- [31] J. Krupp, C. Rossow, TEETHER: Gnawing at Ethereum to automatically exploit smart contracts, in: *Proceedings of the 27th USENIX Security Symposium*, USENIX Association, 2018, pp. 1317–1333.
- [32] J. Chen, X. Xia, D. Lo, et al., DEFECTCHECKER: automated smart contract defect detection by analyzing EVM bytecode, *IEEE Trans. Softw. Eng.* 48 (7) (2022) 2189–2207, <https://doi.org/10.1109/TSE.2021.3054928>.
- [33] J. Li, Metamorphic Testing for Smart Contract Vulnerabilities Detection, *arXiv preprint, arXiv:2303.03179*, 2023.
- [34] H. Wu, X. Wang, J. Xu, et al., Mutation testing for Ethereum smart contract, *arXiv preprint, arXiv:1908.03707*, 2020.
- [35] P. Tolmach, Y. Li, S.-W. Lin, et al., A survey of smart contract formal specification and verification, *arXiv preprint, arXiv:2008.02712*, 2021.
- [36] S.S. Kushwaha, S. Joshi, D. Singh, et al., Ethereum smart contract analysis tools: a systematic review, *IEEE Access* 10 (2024) 57037–57062, <https://doi.org/10.1109/ACCESS.2022.3169902>.
- [37] Z. Zhang, B. Zhang, W. Xu, et al., Demystifying exploitable bugs in smart contracts, in: *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, IEEE, 2023, pp. 615–627, <https://doi.org/10.1109/ICSE48619.2023.00061>.
- [38] F.R. Vidal, N. Ivaki, N. Laranjeiro, OpenSCV: an open hierarchical taxonomy for smart contract vulnerabilities, *arXiv preprint, arXiv:2303.14523*, 2023.
- [39] ChainSecurity, Securiify is now github, <https://medium.com/chainsecurity/securiify-is-nowon-github-d3bec281eafc>, 2018. (Accessed 15 August 2023).
- [40] Kamilpolak, Hack solidity: reentrancy attack, <https://hackernoon.com/hack-solidity-reentrancy-attack>, 2022. (Accessed 15 August 2023).
- [41] Reentrancy attack in a solidity smart contract, <https://cryptomarketpool.com/reentrancy-attack-in-a-solidity-smart-contract/>, 2024. (Accessed 15 August 2023).
- [42] R. Singhal, Recognizing reentrancy attacks in Ethereum smart contracts, <https://doi.org/10.13140/RG.2.2.29494.78409>, 2022. (Accessed 15 August 2023).
- [43] N.D.W. Foxley, Weekend attack drains decentralized protocol dforce of \$25m in crypto, <https://www.coindesk.com/markets/2020/04/19/weekend-attack-drains-decentralized-protocol-dforce-of-25m-in-crypto/>, 2020. (Accessed 15 August 2023).
- [44] M. Yang, A summary of the attack on lendf, <https://medium.com/dforcenet/a-summary-of-the-attack-on-lendf-me-on-april-19-2020-e2f1c5d96640>, 2020. (Accessed 15 August 2023).

- [45] R. Behnke, Explained: the grim finance hack, <https://www.halborn.com/blog/post/explained-the-grim-finance-hack-december-2021>, 2021. (Accessed 15 August 2023).
- [46] S. Kalra, S. Goel, M. Dhawan, et al., ZEUS: analyzing safety of smart contracts, in: Proceedings of the 2018 Network and Distributed System Security Symposium, Internet Society, 2018, <https://doi.org/10.14722/ndss.2018.23082>.
- [47] P. Momeni, Y. Wang, R. Samavi, Machine learning model for smart contracts security analysis, in: Proceedings of the 2019 17th International Conference on Privacy, Security and Trust (PST), IEEE, 2019, pp. 1–6, <https://doi.org/10.1109/PST47121.2019.8949045>.
- [48] Smart Contract Security, Smart contract weakness classification (SWC) and test cases, <http://swcregistry.io/>, 2020. (Accessed 15 August 2023).
- [49] P. Tsankov, A. Dan, D. Drachler-Cohen, et al., Securify: practical security analysis of smart contracts, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18, ACM, 2018, pp. 67–82, <https://doi.org/10.1145/3243734.3243780>.
- [50] A. Mavridou, A. Laszka, Designing secure Ethereum smart contracts: a finite state machine based approach, in: S. Meiklejohn, K. Sako (Eds.), Financial Cryptography and Data Security, Springer, Berlin, 2018, pp. 523–540, https://doi.org/10.1007/978-3-662-58387-6_28.
- [51] B. Jiang, Y. Liu, W. Chan, ContractFuzzer: fuzzing smart contracts for vulnerability detection, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ACM, 2018, pp. 259–269, <https://doi.org/10.1145/3238147.3238177>.
- [52] T. Nguyen, L. Pham, J. Sun, et al., sFuzz: an efficient adaptive fuzzer for solidity smart contracts, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, ICSE '20, ACM, 2020, pp. 778–788, <https://doi.org/10.1145/3377811.3380334>.
- [53] E. Andesta, F. Faghni, M. Fooladgar, Testing smart contracts gets smarter, in: Proceedings of the 2020 10th International Conference on Computer and Knowledge Engineering (ICCKE), IEEE, 2020, pp. 405–412, <https://doi.org/10.1109/ICCKE50421.2020.9303670>.
- [54] X. Li, X. Xing, G. Wang, et al., Detecting unknown vulnerabilities in smart contracts with binary classification model using machine learning, in: G. Wang, K.K.R. Choo, J. Wu, et al. (Eds.), Ubiquitous Security, Springer, Singapore, 2022, pp. 179–192, https://doi.org/10.1007/978-981-99-0272-9_12.
- [55] T. Hu, B. Li, Z. Pan, et al., Detect defects of solidity smart contract based on the knowledge graph, IEEE Trans. Reliab. 73 (1) (2023) 186–202, <https://doi.org/10.1109/TR.2023.3233999>.
- [56] S. Cui, G. Zhao, Y. Gao, et al., VRust: automated vulnerability detection for Solana smart contracts, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2022, pp. 639–652, <https://doi.org/10.1145/3548606.3560552>.
- [57] L. Brent, N. Grech, S. Lagouvardos, et al., Ethainter: a smart contract security analyzer for composite vulnerabilities, in: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2020, pp. 454–469, <https://doi.org/10.1145/3385412.3385990>.
- [58] M.C. Argañaraz, M.M. Berón, M.J.V. Pereira, et al., Detection of vulnerabilities in smart contracts specifications in Ethereum platforms, in: Proceedings of the 9th Symposium on Languages, Applications and Technologies (SLATE 2020), Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020, pp. 2:1–2:16, <https://doi.org/10.4230/OASICS.SLATE.2020.2>.
- [59] Q. Zhang, Y. Wang, J. Li, et al., Ethploit: from fuzzing to efficient exploit generation against smart contracts, in: Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2020, pp. 116–126, <https://doi.org/10.1109/SANER48275.2020.9054822>.
- [60] F. Ma, Z. Xu, M. Ren, et al., Pluto: exposing vulnerabilities in inter-contract scenarios, IEEE Trans. Softw. Eng. 48 (11) (2022) 4380–4396, <https://doi.org/10.1109/TSE.2021.3117966>.
- [61] T. Chen, R. Cao, T. Li, et al., SODA: a generic online detection framework for smart contracts, in: Proceedings of the 2020 Network and Distributed System Security Symposium, Internet Society, 2020, <https://www.ndss-symposium.org/wp-content/uploads/2020/02/24449.pdf>.
- [62] J.W. Liao, T.T. Tsai, C.K. He, et al., SoliAudit: smart contract vulnerability assessment based on machine learning and fuzz testing, in: Proceedings of the 2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS), IEEE, 2019, pp. 458–465, <https://doi.org/10.1109/IOTSMS48152.2019.8939256>.
- [63] A. Kolluri, I. Nikolic, I. Sergey, et al., Exploiting the laws of order in smart contracts, in: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019, ACM, 2019, pp. 363–373, <https://doi.org/10.1145/3293882.3330560>.
- [64] J. Stephens, K. Ferles, B. Mariano, et al., SmartPulse: automated checking of temporal properties in smart contracts, in: Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP), IEEE, 2021, pp. 555–571, <https://doi.org/10.1109/SP40001.2021.00085>.
- [65] H. Wang, Y. Li, S.W. Lin, et al., VULTRON: catching vulnerable smart contracts once and for all, in: Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), IEEE, 2019, pp. 1–4, <https://doi.org/10.1109/ICSE-NIER.2019.00009>.
- [66] C.F. Torres, J. Schütte, R. State, Osiris: hunting for integer bugs in Ethereum smart contracts, in: Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC '18), ACM, 2018, pp. 664–676, <https://doi.org/10.1145/3274694.3274737>.
- [67] M. Jurgelaitis, L. Čeponienė, K. Butkus, et al., MDA-based approach for blockchain smart contract development, Appl. Sci. 13 (1) (2023) 487, <https://doi.org/10.3390/app13010487>.
- [68] S. Dharanikota, S. Mukherjee, C. Bhardwaj, et al., Celestial: a smart contracts verification framework, in: Proceedings of the 2021 Formal Methods in Computer Aided Design (FMCAD), IEEE, 2021, pp. 133–142, https://doi.org/10.34727/2021/isbn.978-3-85448-046-4_22.
- [69] Y. Ucbas, A. Eleyan, M. Hammoudeh, et al., Performance and scalability analysis of Ethereum and Hyperledger Fabric, IEEE Access 11 (2023) 67156–67167, <https://doi.org/10.1109/ACCESS.2023.3291618>.
- [70] M. Ceresa, C. Sánchez, Multi: a formal playground for multi-smart contract interaction, arXiv preprint, arXiv:2207.06681, 2022.
- [71] Bloqarl, Security system starts with the testing: how to properly battle test your smart contracts, <https://medium.com/rektoff/a-security-system-starts-with-the-testing-how-to-properly-battle-test-your-smart-contracts-4dd3a7538959>, 2023. (Accessed 15 August 2023).